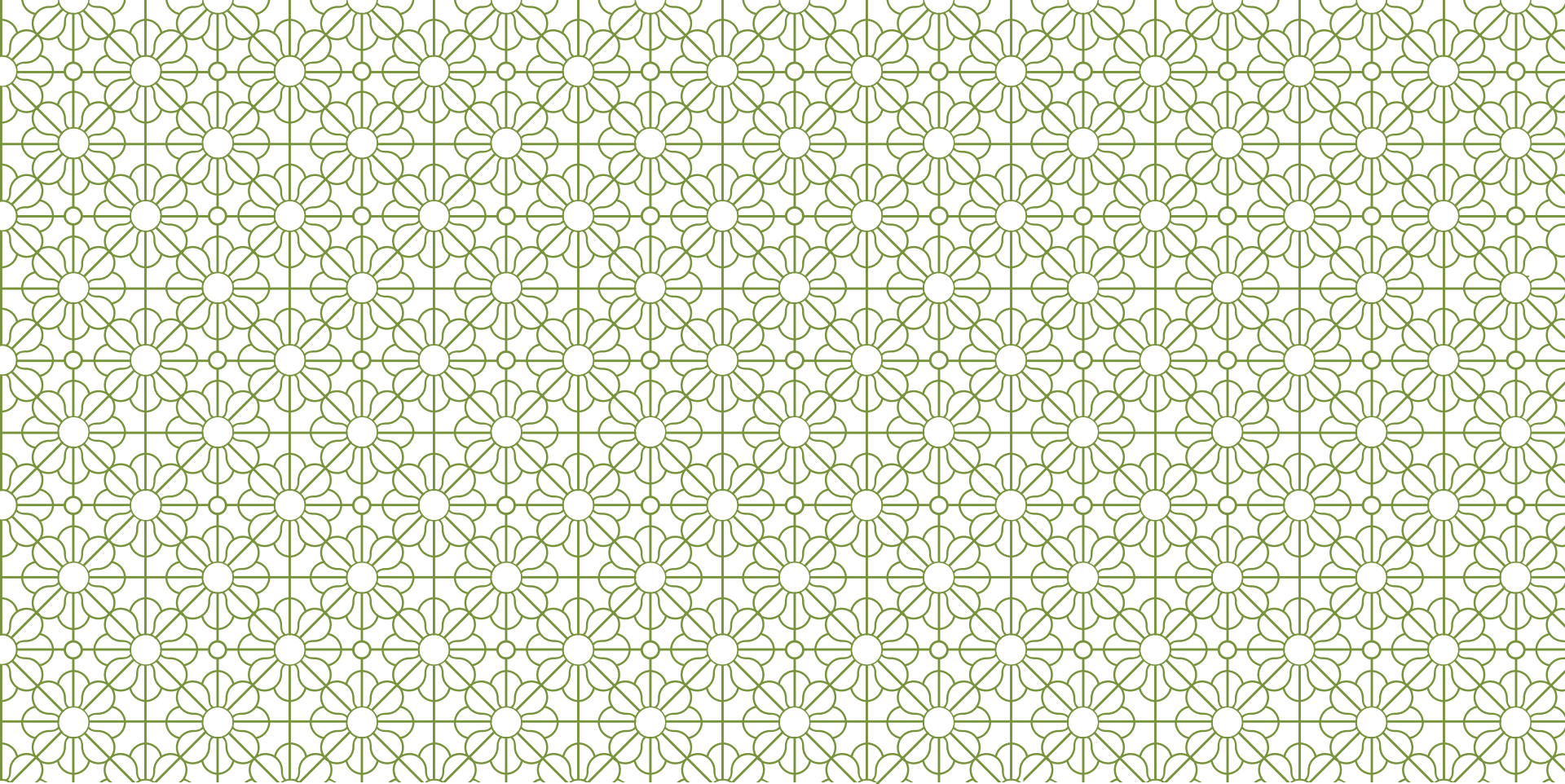




# ARQUITETURA DE COMPUTADORES

Arquiteturas VLIW

# ARQUITETURAS VLIW

Introdução

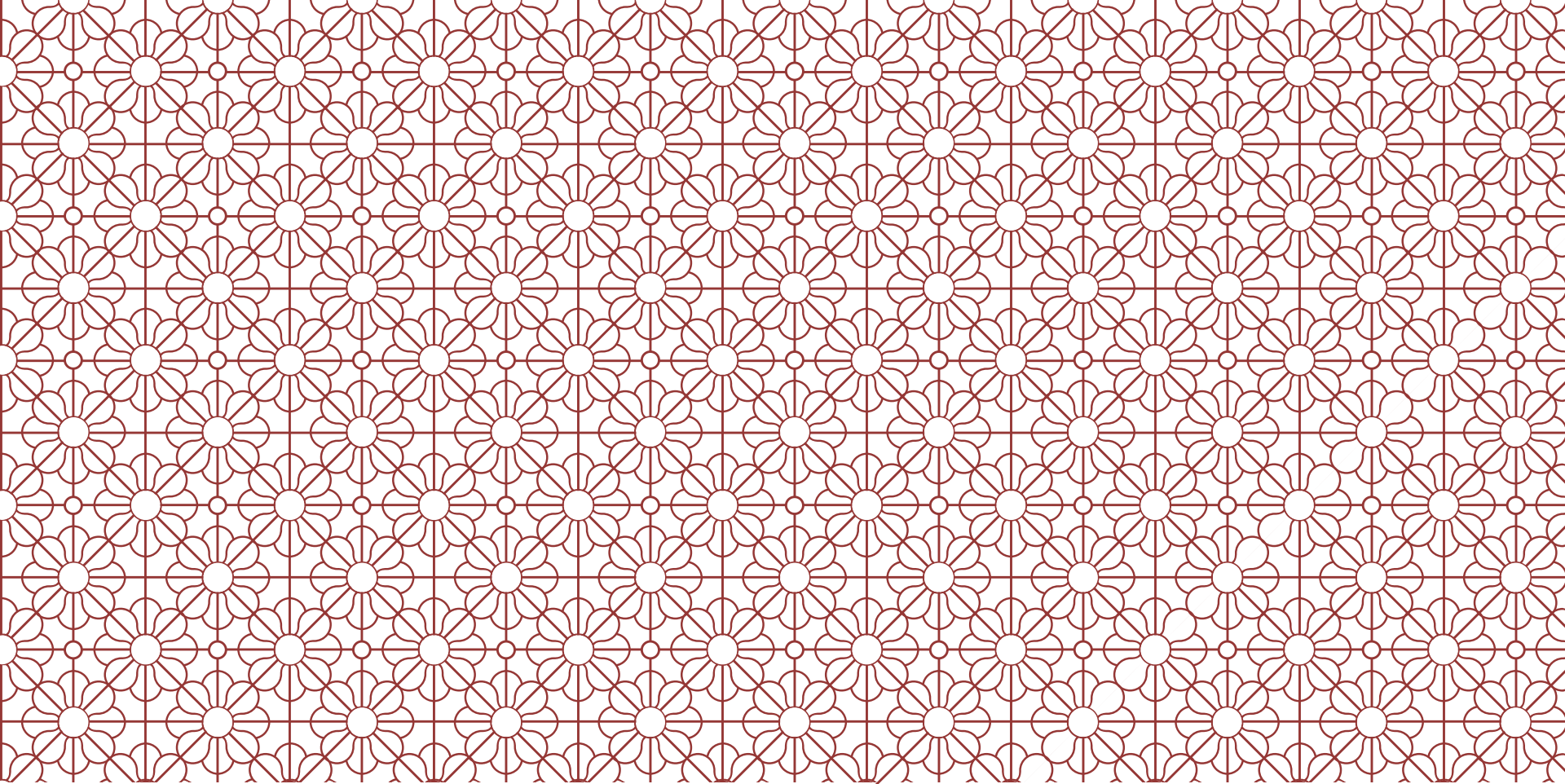
Escalonamento

Escalonamento acíclico

Escalonamento cíclico

Exercício

Processador Itanium



# CONTEXTO HISTÓRICO

# CONTEXTO HISTÓRICO

O que estava ocorrendo em 2000?

# CONTEXTO HISTÓRICO





# CONTEXTO HISTÓRICO

## Técnicas agressivas de paralelismo de instruções (ILP)

- Reorder buffer aumentando
- Preditores de salto ficando mais sofisticados
- Escalonadores complexos

## Frequência de operação aumentando

- Pipelines se tornando cada vez mais profundos
- Maior consumo de energia e temperatura do chip
- Controle cada vez mais complexo

# CONTEXTO HISTÓRICO

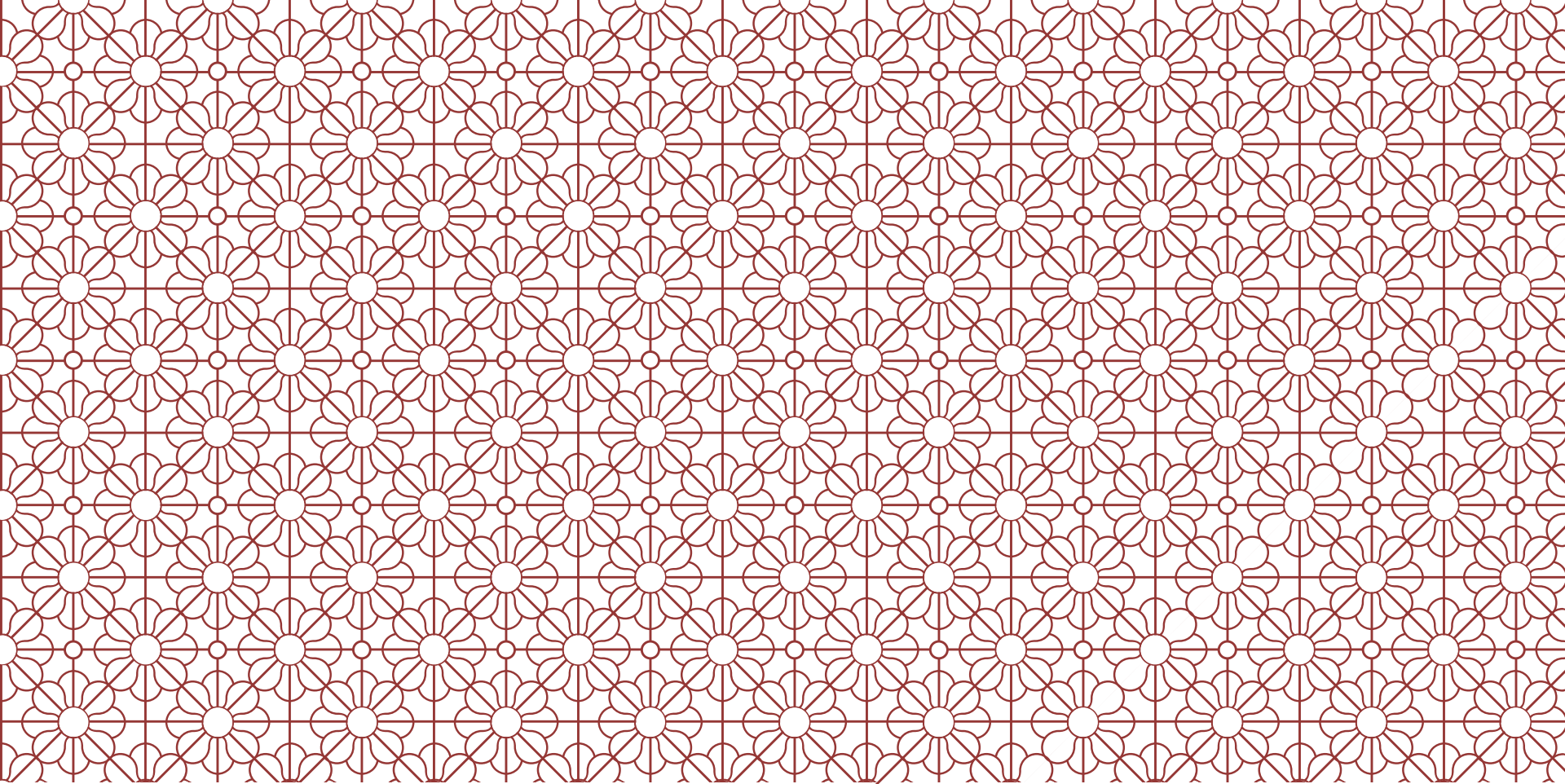
## Técnicas agressivas de paralelismo de instruções (ILP)

- Reorder buffer aumentando
- Preditores de salto ficando mais sofisticados
- Escalonadores complexos

## Frequência de operação aumentando

- Pipelines se tornando cada vez mais profundos
- Maior consumo de energia e temperatura do chip
- Controle cada vez mais complexo

E se transferíssemos  
toda complexidade de  
**encontrar instruções  
paralelas** para o  
software?



# VLIW

## VERY LONG INSTRUCTION WORD



# INTRODUÇÃO AO VLIW

## **VLIW - Very Long Instruction Word**

Exploram paralelismo no nível das instruções (ILP)

Várias operações são executadas em paralelo em diferentes unidades funcionais, tais como em máquinas superescalares

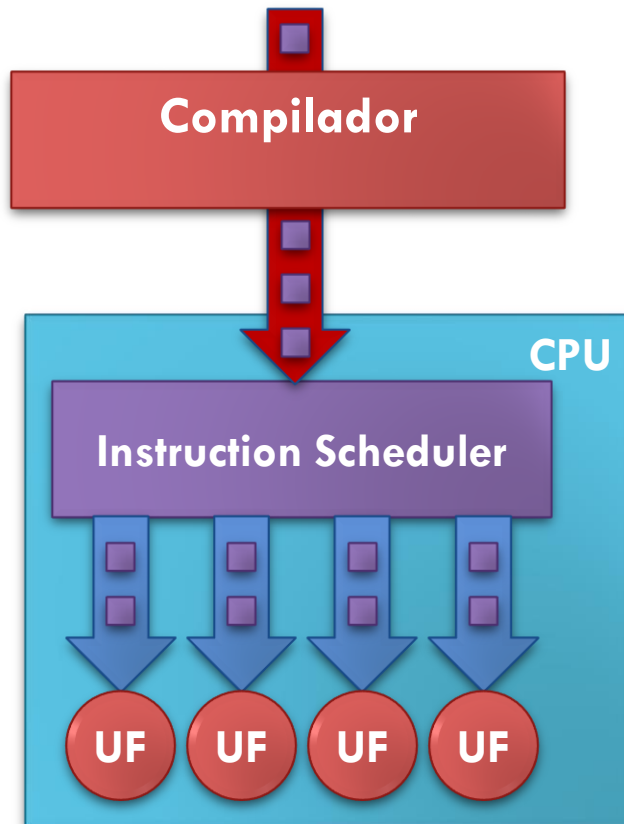
Diferença no despacho e da graduação das operações:

**Superescalares:** as dependências são resolvidas em tempo de execução por um hardware dedicado

**VLIW:** as dependências são resolvidas em tempo de compilação

# SUPERESCALAR VS VLIW

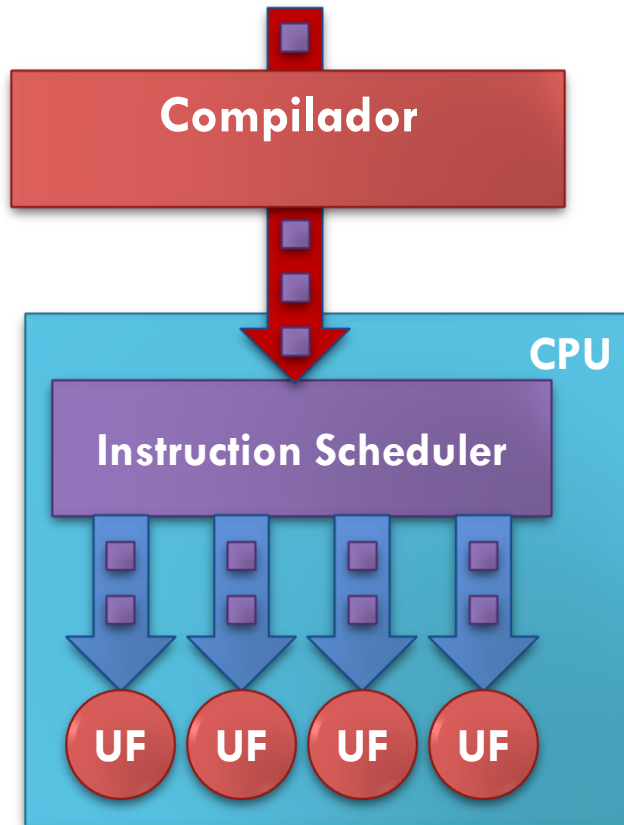
Superescalar



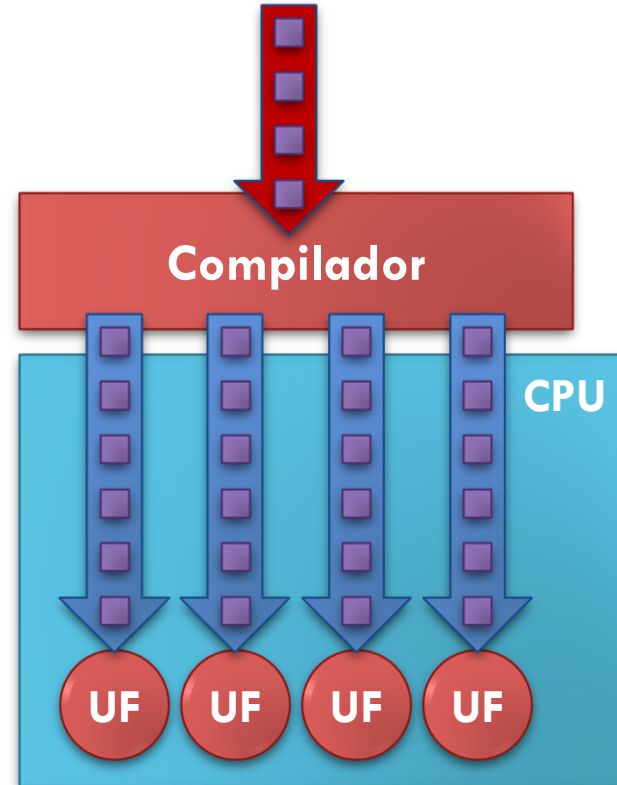
# SUPERESCALAR VS VLIW

Continuamos com 1 única thread.  
Mas a instrução é enorme, já  
mapeada para unidades funcionais

## Superescalar



## VLIW



# INTRODUÇÃO (2)

No VLIW, várias instruções são codificadas em uma mesma instrução

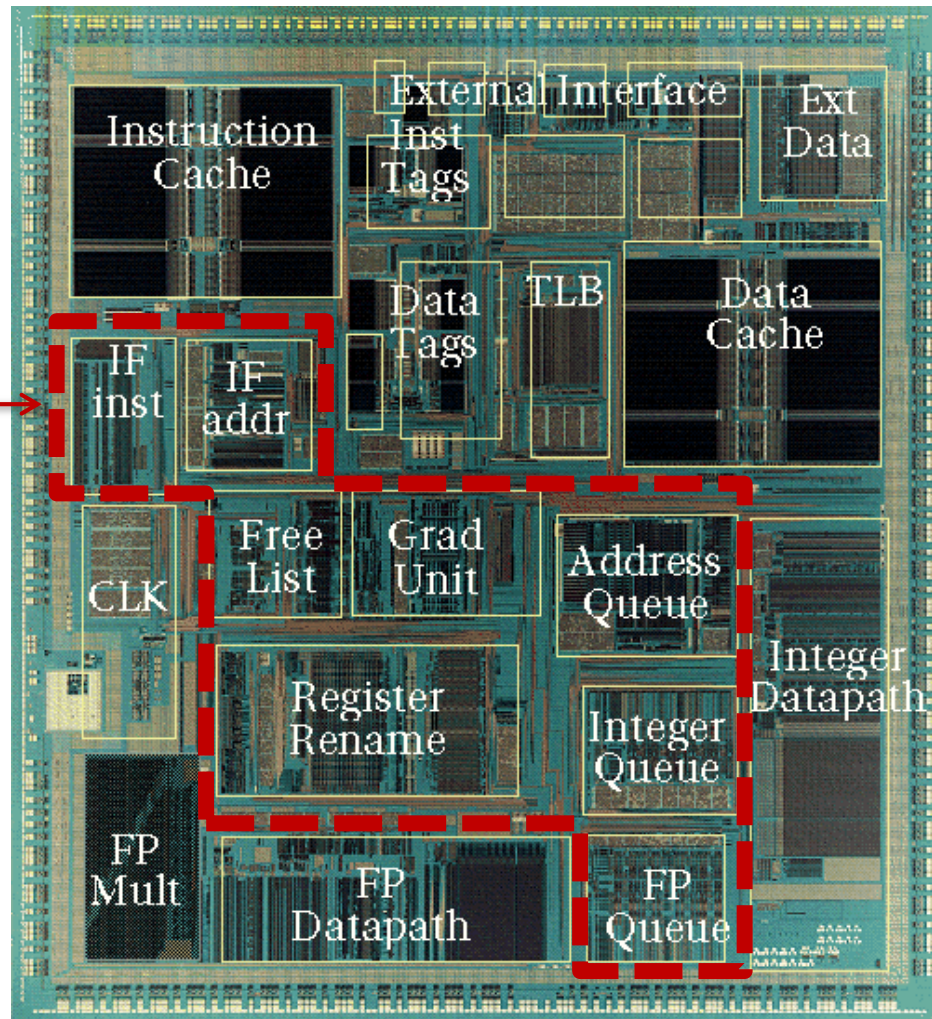
A palavra de instrução é bastante longa, podendo conter várias operações independentes

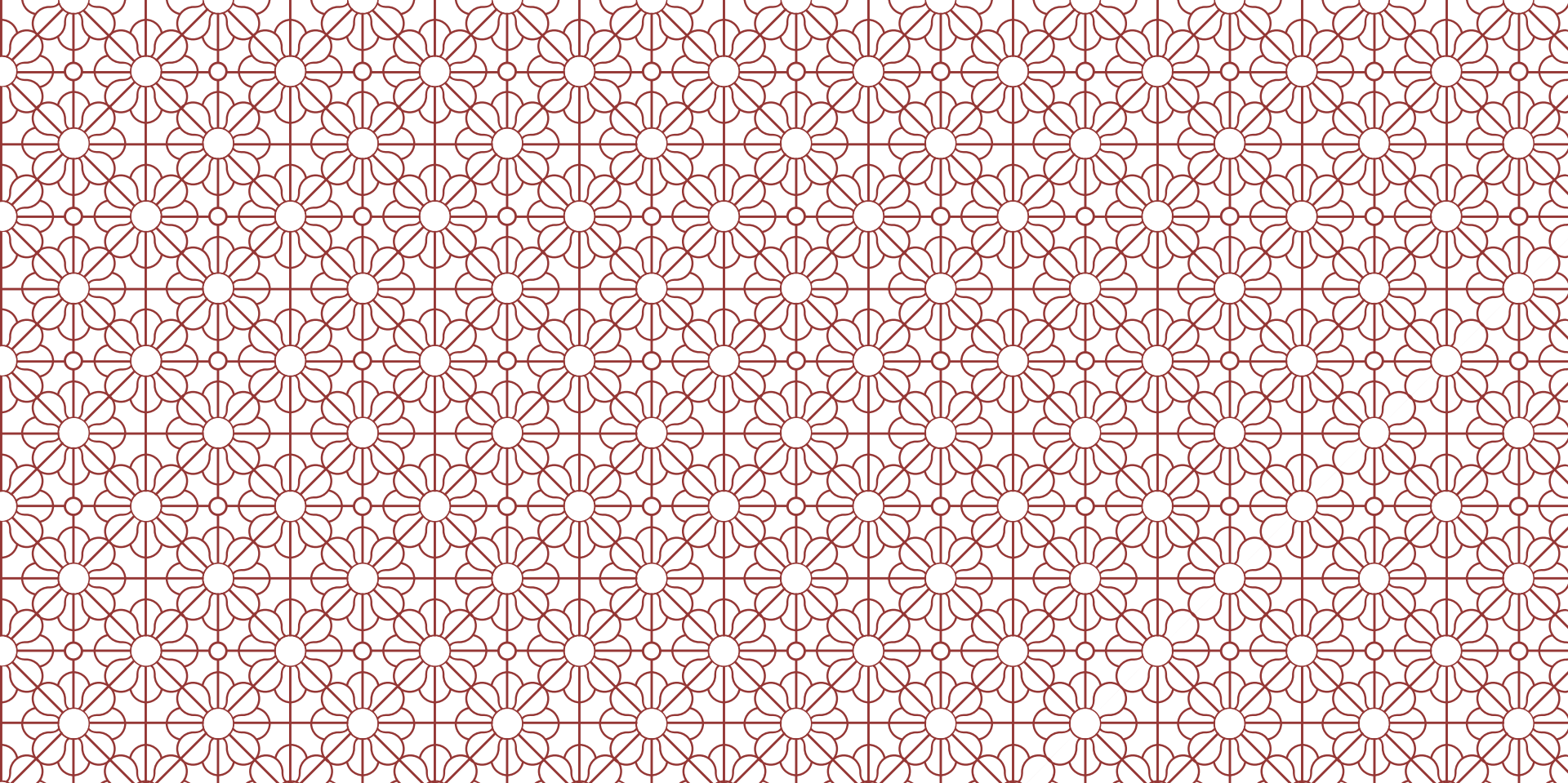
A posição de cada operação dentro da palavra VLIW determina a unidade funcional que será usada

O hardware de despacho é simples

# COMPLEXIDADE: 000 MIPS R10000

**Lógica de  
controle e  
escalonamento**





# 0 PIPELINE VLIW



# CONCEITO BÁSICO DE VLIW

Uso de múltiplas unidades funcionais independentes

As operações são executadas em paralelo nessas unidades funcionais

Para isso, são empacotadas em 1 única instrução VLIW

- Grupo de operações que serão executadas em paralelo
- 1 instrução possui múltiplas operações concorrentes
- Slots de operação não utilizados são preenchidos com NOPs

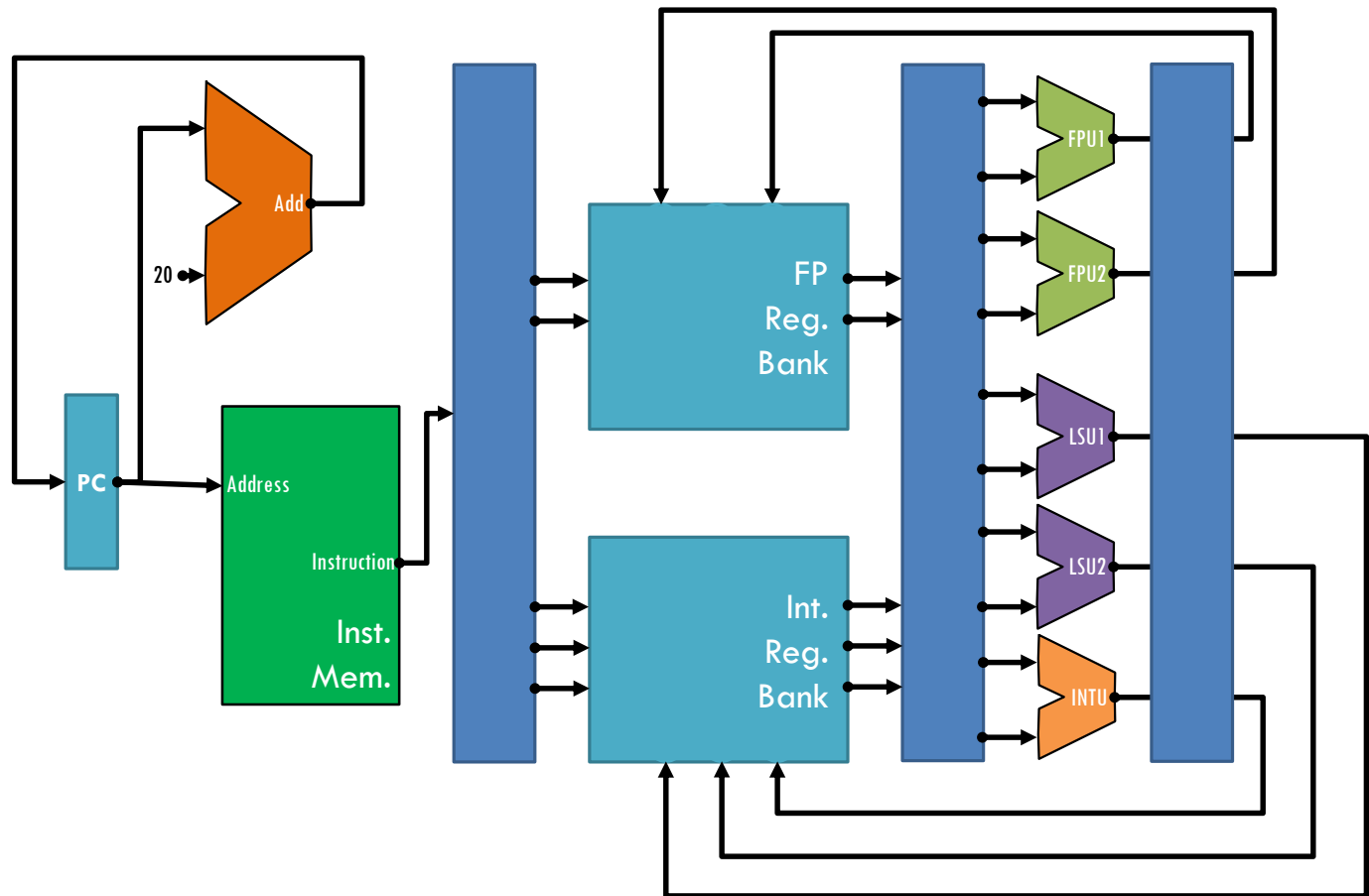
Exemplo de uma instrução VLIW com as seguintes unidades:

2 LD/ST, 2 FPU, 1 INT/Branch



# EXEMPLO DE UM PROCESSADOR VLIW PIPELINED

LD/ST Op1   LD/ST Op2   FP Op1   FP Op2   INT/Branch Op



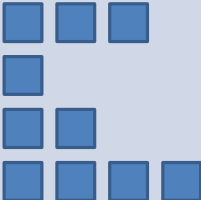
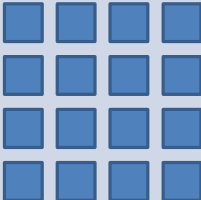
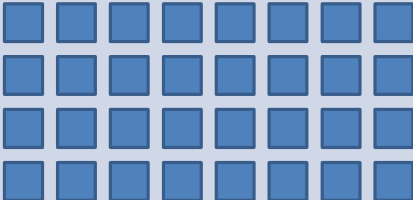
# COMPARAÇÃO

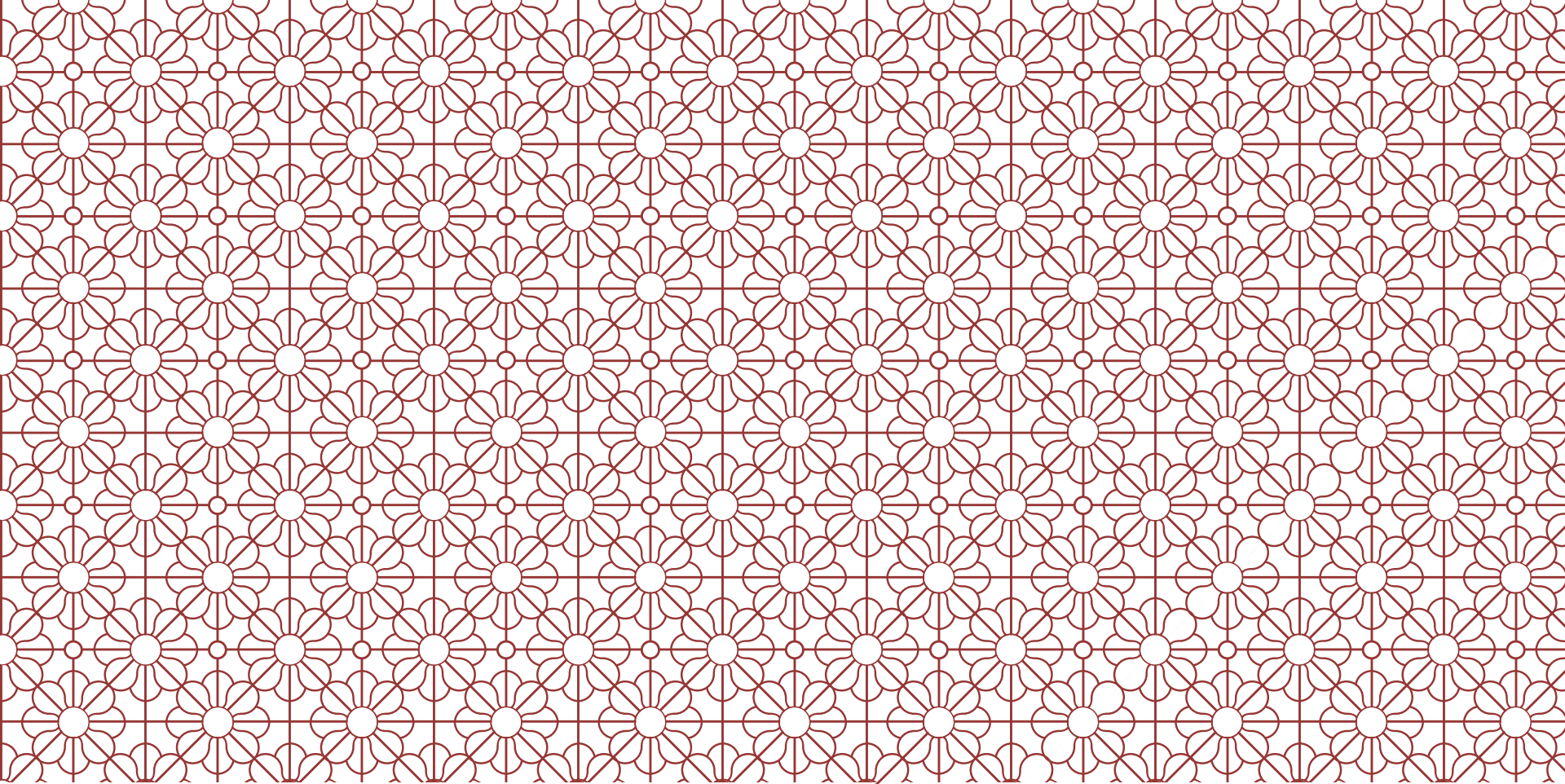
| Característica       | CISC                      | RISC                                  | VLIW                                  |
|----------------------|---------------------------|---------------------------------------|---------------------------------------|
| Tamanho da Instrução | Variável                  | Um tamanho                            | Um tamanho                            |
| Formato da Instrução | Local dos campos variável | Regular, local dos campos consistente | Regular, local dos campos consistente |

# COMPARAÇÃO

| Característica         | CISC                                                                          | RISC                                                     | VLIW                                                     |
|------------------------|-------------------------------------------------------------------------------|----------------------------------------------------------|----------------------------------------------------------|
| Tamanho da Instrução   | Variável                                                                      | Um tamanho                                               | Um tamanho                                               |
| Formato da Instrução   | Local dos campos variável                                                     | Regular, local dos campos consistente                    | Regular, local dos campos consistente                    |
| Semântica da Instrução | Desde simples a complexa;<br>Com diversas operações dependentes por instrução | Quase sempre uma operação simples                        | Várias operações simples e independentes                 |
| Registradores          | Poucos, algumas vezes especiais                                               | Vários, propósito geral                                  | Vários, propósito geral                                  |
| Referências de Memória | Agregadas com operações e vários diferentes tipos de instruções               | Não agregadas com operações, i.e. arquitetura load/store | Não agregadas com operações, i.e. arquitetura load/store |

# COMPARAÇÃO

| Característica                                         | CISC                                                                                | RISC                                                                                 | VLIW                                                                                  |
|--------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Tamanho da Instrução                                   | Variável                                                                            | Um tamanho                                                                           | Um tamanho                                                                            |
| Formato da Instrução                                   | Local dos campos variável                                                           | Regular, local dos campos consistente                                                | Regular, local dos campos consistente                                                 |
| Semântica da Instrução                                 | Desde simples a complexa;<br>Com diversas operações dependentes por instrução       | Quase sempre uma operação simples                                                    | Várias operações simples e independentes                                              |
| Registradores                                          | Poucos, algumas vezes especiais                                                     | Vários, propósito geral                                                              | Vários, propósito geral                                                               |
| Referencias de Memória                                 | Agregadas com operações e vários diferentes tipos de instruções                     | Não agregadas com operações, i.e. arquitetura load/store                             | Não agregadas com operações, i.e. arquitetura load/store                              |
| Foco no design do hardware                             | Explorar micro-códigos                                                              | Explorar um pipeline sem micro-código                                                | Explorar múltiplos pipelines, sem micro-código e lógica simples de despacho.          |
| Figura de quatro instruções típicas (blocos de 1 byte) |  |  |  |



# COMPILANDO PARA VLIW



# FLUXO DE EXECUÇÃO

Normalmente a compilação de um programa é feita em três fases

- Parsing da linguagem de entrada para uma estrutura intermediária
- Otimização da estrutura intermediária
- Geração do código para a arquitetura-alvo

A estrutura de dados intermediária representa:

- Fluxo de controle – As diferentes sequências de execução das instruções que formam o programa
- Fluxo de dados – As dependências de dados que existem entre as várias operações

Blocos básicos: grupos de instruções que são SEMPRE executadas em sequência (ou seja, bloco de instruções com apenas 1 ponto entrada e 1 de saída)

# BLOCOS BÁSICOS

```
1) i := m - 1
2) j := n
3) t1 := 4 * n
4) v := a[t1]
5) i := i + 1
6) t2 := 4 * i
7) t3 := a[t2]
8) if t3 < v goto 5)
9) j := j - 1
10) t4 := 4 * j
11) t5 := a[t4]
12) if t5 > v goto 9)
13) if i >= j goto 23)
14) t6 := 4 * i
15) x := a[t6]
```

...

Onde estão os blocos básicos, ou seja, bloco de instruções com apenas 1 ponto entrada e 1 de saída?



# BLOCOS BÁSICOS

```
1) i := m - 1
2) j := n
3) t1 := 4 * n
4) v := a[t1]
5) i := i + 1
6) t2 := 4 * i
7) t3 := a[t2]
8) if t3 < v goto 5)
9) j := j - 1
10) t4 := 4 * j
11) t5 := a[t4]
12) if t5 > v goto 9)
13) if i >= j goto 23)
14) t6 := 4 * i
15) x := a[t6]
```

Podemos mudar a ordem das instruções, desde que elas pertençam ao mesmo Bloco Básico!

○ que pode acontecer se migrarmos uma instrução de um bloco para outro?

# ESCALONAMENTO

O escalonamento das operações consiste em determinar as operações que serão executadas em paralelo

O compilador é responsável por esta tarefa para VLIW

Operações que podem ser executadas em paralelo são atribuídas à mesma palavra de instrução

Em um mesmo bloco básico estas instruções podem ser escalonadas com base no fluxo de dados

- Exemplo: escalonamento ASAP (as soon as possible)

# ESCALONAMENTO DENTRO DE UM BLOCO BÁSICO

;; Sequência de Instruções

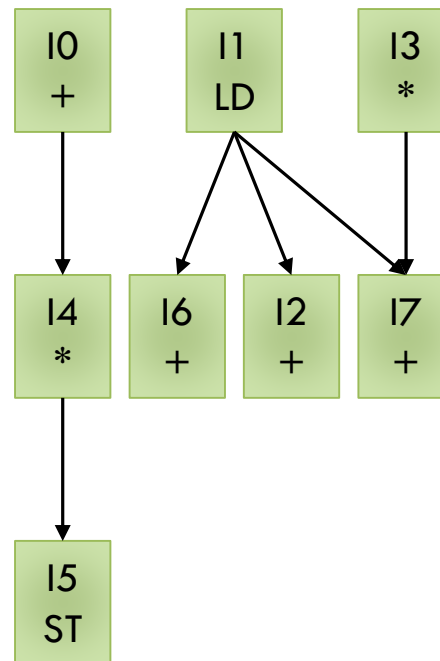
```
I0      R1 <- R2 + R3
I1      R4 <- M[X] ;(Load)
I2      R5 <- R4 + R3
I3      R6 <- R2 * R3
I4      R7 <- R1 * R3
I5      M[X] <- R7 ;(Store)
I6      R8 <- R4 + R3
I7      R9 <- R6 + R4
```

# ESCALONAMENTO DENTRO DE UM BLOCO BÁSICO

## ;;Sequência de Instruções

```
I0      R1 <- R2 + R3
I1      R4 <- M[X] ;(Load)
I2      R5 <- R4 + R3
I3      R6 <- R2 * R3
I4      R7 <- R1 * R3
I5      M[X] <- R7 ;(Store)
I6      R8 <- R4 + R3
I7      R9 <- R6 + R4
```

## Grafo Dataflow



# ESCALONAMENTO DENTRO DE UM BLOCO BÁSICO

## ;; Sequência de Instruções

```

I0      R1 <- R2 + R3
I1      R4 <- M[X] ; (Load)
I2      R5 <- R4 + R3
I3      R6 <- R2 * R3
I4      R7 <- R1 * R3
I5      M[X] <- R7 ; (Store)
I6      R8 <- R4 + R3
I7      R9 <- R6 + R4
    
```

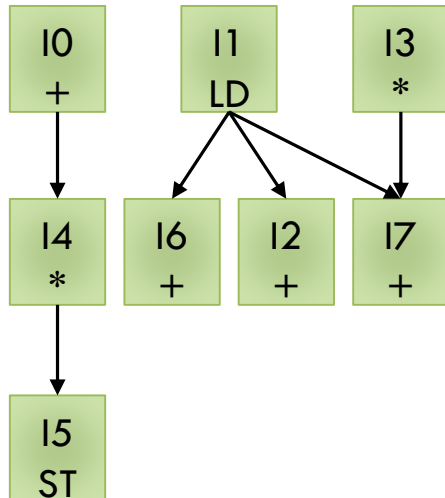
### Latências:

2x Adder = 1 ciclo

1x Multiplier = 3 ciclos (pipelined)

1x Load = 2 ciclos (pipeline)

1x Store = 1 ciclo



## Codificação VLIW

| Somador |         |         | Somador |         |         | Mult.   |         |         | Load    |          | Store    |        |
|---------|---------|---------|---------|---------|---------|---------|---------|---------|---------|----------|----------|--------|
| DESTINO | ORIGEM1 | ORIGEM2 | DESTINO | ORIGEM1 | ORIGEM2 | DESTINO | ORIGEM1 | ORIGEM2 | DESTINO | ENDEREÇO | ENDEREÇO | ORIGEM |
|         |         |         |         |         |         |         |         |         |         |          |          |        |
|         |         |         |         |         |         |         |         |         |         |          |          |        |
|         |         |         |         |         |         |         |         |         |         |          |          |        |
|         |         |         |         |         |         |         |         |         |         |          |          |        |
|         |         |         |         |         |         |         |         |         |         |          |          |        |
|         |         |         |         |         |         |         |         |         |         |          |          |        |
|         |         |         |         |         |         |         |         |         |         |          |          |        |



# ESCALONAMENTO DENTRO DE UM BLOCO BÁSICO

## ;; Sequência de Instruções

```

I0      R1 <- R2 + R3
I1      R4 <- M[X] ; (Load)
I2      R5 <- R4 + R3
I3      R6 <- R2 * R3
I4      R7 <- R1 * R3
I5      M[X] <- R7 ; (Store)
I6      R8 <- R4 + R3
I7      R9 <- R6 + R4
    
```

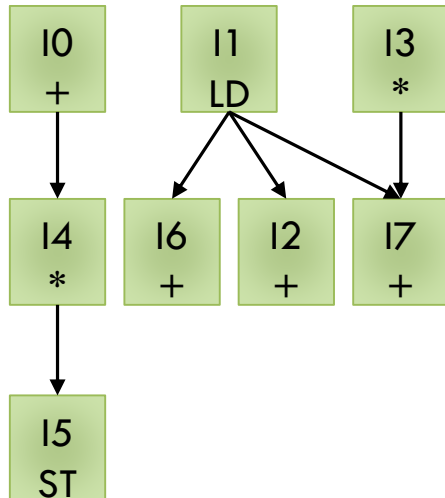
### Latências:

2x Adder = 1 ciclo

1x Multiplier = 3 ciclos (pipelined)

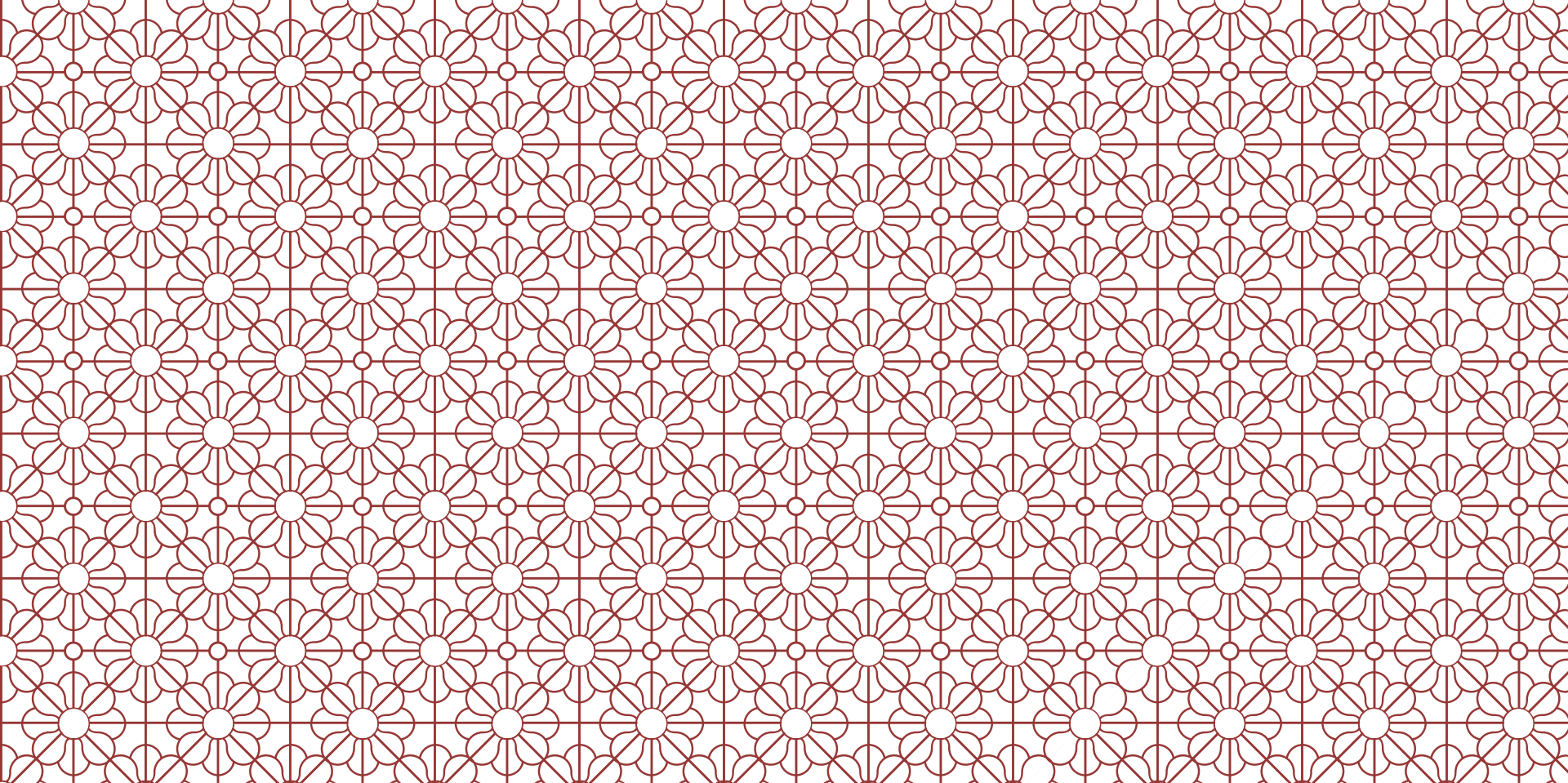
1x Load = 2 ciclos (pipeline)

1x Store = 1 ciclo



## Codificação VLIW

| Somador |         |         | Somador |         |         | Mult.   |         |         | Load    |          | Store    |        |
|---------|---------|---------|---------|---------|---------|---------|---------|---------|---------|----------|----------|--------|
| DESTINO | ORIGEM1 | ORIGEM2 | DESTINO | ORIGEM1 | ORIGEM2 | DESTINO | ORIGEM1 | ORIGEM2 | DESTINO | ENDEREÇO | ENDEREÇO | ORIGEM |
| R1      | R2      | R3      |         |         |         | R6      | R2      | R3      | R4      | X        |          |        |
|         |         |         |         |         |         | R7      | R1      | R3      |         |          |          |        |
| R5      | R4      | R3      | R8      | R4      | R3      |         |         |         |         |          |          |        |
| R9      | R6      | R4      |         |         |         |         |         |         |         |          |          |        |
|         |         |         |         |         |         |         |         |         |         |          |          |        |
|         |         |         |         |         |         |         |         |         |         |          |          |        |
|         |         |         |         |         |         |         |         |         |         |          |          |        |



# EXERCÍCIO



# EXERCÍCIO ESCALONAMENTO

Considerando 2 unidades Soma, 2 Multiplicadores, 1 LD/ST

Faça o escalonamento ASAP (as soon as possible)

```
1) i := m - 1
2) j := n
3) t1 := 4 * n
4) v := a[t1]
5) i := i + 1
6) t2 := 4 * i
7) t3 := a[t2]
8) if t3 < v goto 5)
```

Comece por um  
grafo de  
dependências!

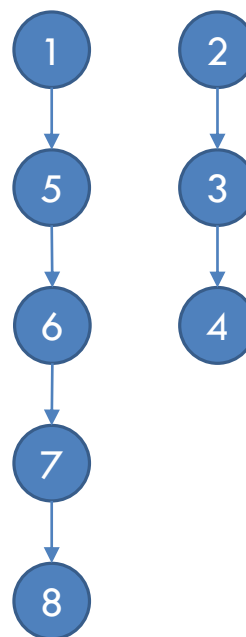


# EXERCÍCIO ESCALONAMENTO

Considerando 2 unidades Soma, 2 Multiplicadores, 1 LD/ST

Faça o escalonamento ASAP (as soon as possible)

```
1) i := m - 1  
2) n := j  
3) t1 := 4 * n  
4) v := a[t1]  
5) i := i + 1  
6) t2 := 4 * i  
7) t3 := a[t2]  
8) if t3 < v goto 5)
```





# EXERCÍCIO ESCALONAMENTO

Considerando 2 unidades Soma, 2 Multiplicadores, 1 LD/ST

Faça o escalonamento ASAP (as soon as possible)

```
1) i := m - 1
2) n := j
3) t1 := 4 * n
4) v := a[t1]
5) i := i + 1
6) t2 := 4 * i
7) t3 := a[t2]
8) if t3 < v goto 5)
```

| Sum | Sum | Mul | Mul | Ld/St |
|-----|-----|-----|-----|-------|
|     |     |     |     |       |
|     |     |     |     |       |
|     |     |     |     |       |
|     |     |     |     |       |
|     |     |     |     |       |
|     |     |     |     |       |
|     |     |     |     |       |
|     |     |     |     |       |

Quais blocos básicos?  
Quais dependências entre instruções?  
Qual a latência por instrução?



# EXERCÍCIO ESCALONAMENTO

Considerando 2 unidades Soma, 2 Multiplicadores, 1 LD/ST

Faça o escalonamento ASAP (as soon as possible)

```
1) i := m - 1
2) n := j
3) t1 := 4 * n
4) v := a[t1]
5) i := i + 1
6) t2 := 4 * i
7) t3 := a[t2]
8) if t3 < v goto 5)
```

| Sum | Sum | Mul | Mul | Ld/St |
|-----|-----|-----|-----|-------|
| 1   | 2   |     |     |       |
| 5   |     | 3   |     |       |
|     |     | 6   |     | 4     |
|     |     |     |     | 7     |
| 8   |     |     |     |       |
|     |     |     |     |       |
|     |     |     |     |       |

Qual o  
problema?

# EXERCÍCIO ESCALONAMENTO

Considerando 2 unidades Soma, 2 Multiplicadores, 1 LD/ST

Faça o escalonamento ASAP (as soon as possible)

```
1) i := m - 1
2) n := j
3) t1 := 4 * n
4) v := a[t1]
5) i := i + 1
6) t2 := 4 * i
7) t3 := a[t2]
8) if t3 < v goto 5)
```

| Sum | Sum | Mul | Mul | Ld/St |
|-----|-----|-----|-----|-------|
| 1   | 2   |     |     |       |
| 5   |     | 3   |     |       |
|     |     | 6   |     | 4     |
|     |     |     |     | 7     |
| 8   |     |     |     |       |
|     |     |     |     |       |
|     |     |     |     |       |

As instruções 3 e 4 vão ser executadas erroneamente se o salto ocorrer!!!

# EXERCÍCIO ESCALONAMENTO

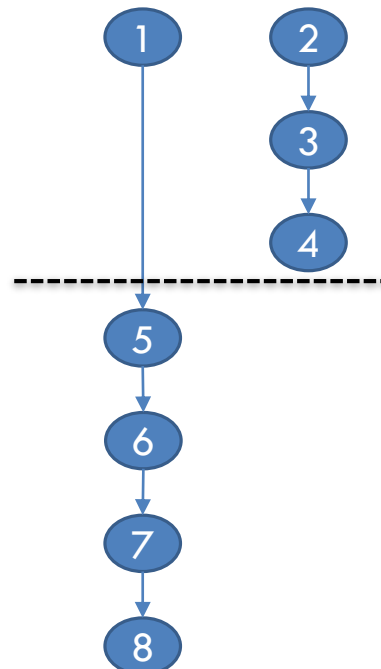
Considerando 2 unidades Soma, 2 Multiplicadores, 1 LD/ST

Faça o escalonamento ASAP (as soon as possible)



```
1) i := m - 1
2) n := j
3) t1 := 4 * n
4) v := a[t1]
-----
5) i := i + 1
6) t2 := 4 * i
7) t3 := a[t2]
8) if t3 < v goto 5)
```

Qual o  
problema?







# EXERCÍCIO ESCALONAMENTO

Considerando 2 unidades Soma, 2 Multiplicadores, 1 LD/ST

Faça o escalonamento ASAP (as soon as possible)

1)  $i := m - 1$   
2)  $n := j$   
3)  $t1 := 4 * n$   
4)  $v := a[t1]$   
-----  
5)  $i := i + 1$   
6)  $t2 := 4 * i$   
7)  $t3 := a[t2]$   
8) if  $t3 < v$  goto 5)

| Sum | Sum | Mul | Mul | Ld/St |
|-----|-----|-----|-----|-------|
| 1   | 2   |     |     |       |
|     |     | 3   |     |       |
|     |     |     |     | 4     |
| 5   |     |     |     |       |
|     |     | 6   |     |       |
|     |     |     |     | 7     |
| 8   |     |     |     |       |

Qual o  
problema?

# EXERCÍCIO ESCALONAMENTO

Considerando 2 unidades Soma, 2 Multiplicadores, 1 LD/ST

Faça o escalonamento ASAP (as soon as possible)



```
1) i := m - 1
2) n := j
3) t1 := 4 * n
4) v := a[t1]
-----
5) i := i + 1
6) t2 := 4 * i
7) t3 := a[t2]
8) if t3 < v goto 5)
```

| Sum | Sum | Mul | Mul | Ld/St |
|-----|-----|-----|-----|-------|
| 1   | 2   |     |     |       |
|     |     | 3   |     |       |
|     |     |     |     | 4     |
| 5   |     |     |     |       |
|     |     | 6   |     |       |
|     |     |     |     | 7     |
|     |     |     |     |       |

Qual o  
problema?

O desempenho depende de  
instruções independentes!!!

# ESCALONAMENTO

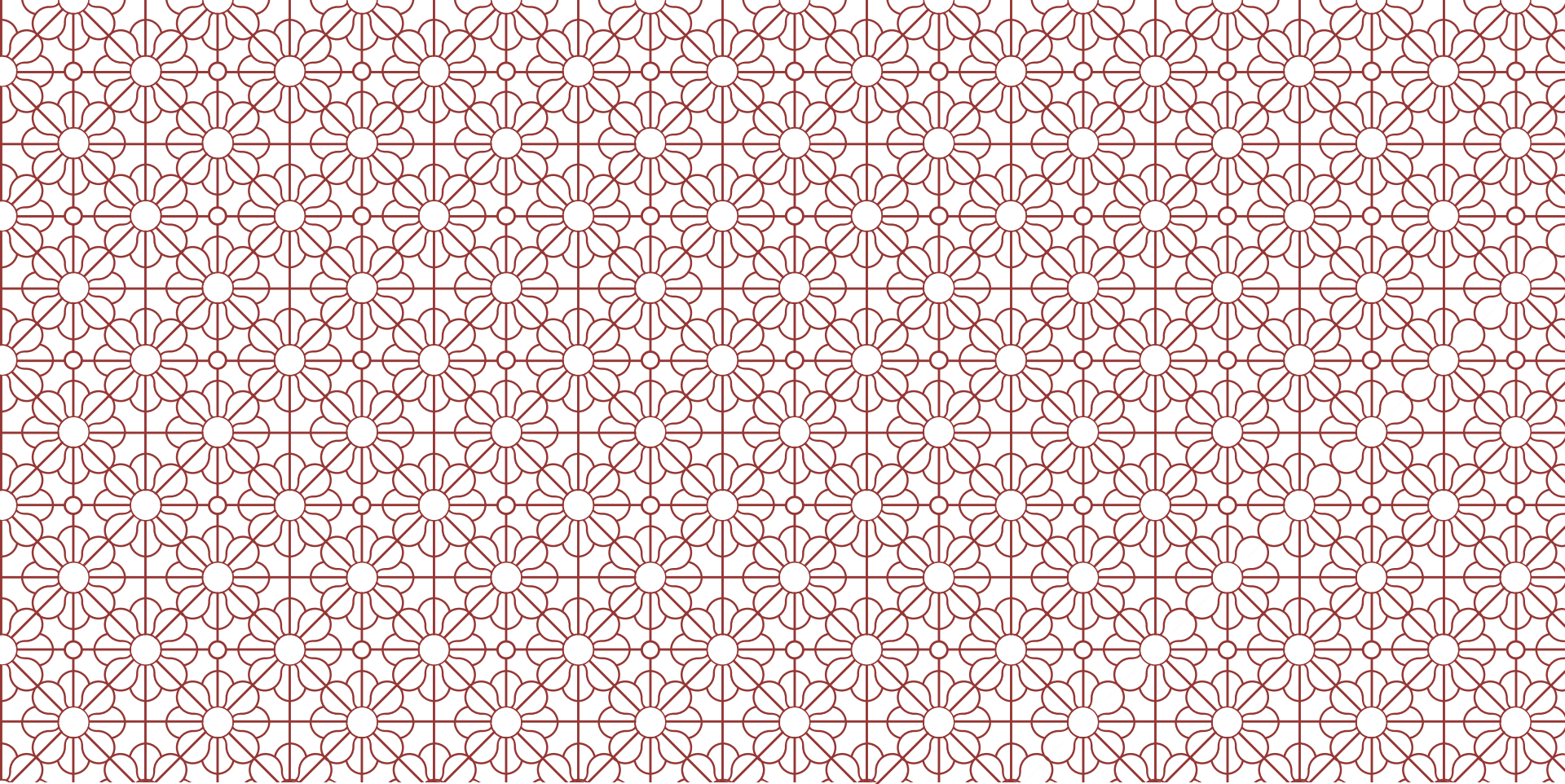
Em um mesmo bloco básico estas instruções podem ser escalonadas com base no fluxo de dados

- Exemplo: escalonamento ASAP (as soon as possible)

A transição entre blocos exige técnicas mais elaboradas:

**Escalonamento acíclico:** programas com desvios, sem ciclos

**Escalonamento cíclico:** programas com ciclos e laços (i.e., causam desvios)



# ESCALONAMENTO ACÍCLICO (PREDICAÇÃO)

# ESCALONAMENTO ACÍCLICO BASEADO EM PREDICADOS

Operações que se seguem a uma condicional são associadas com um predicado

- Predicado é definido em função do resultado da condicional

A palavra VLIW deve ser acrescida de um registrador de predicado

A máquina deve ter uma unidade operativa para predicados

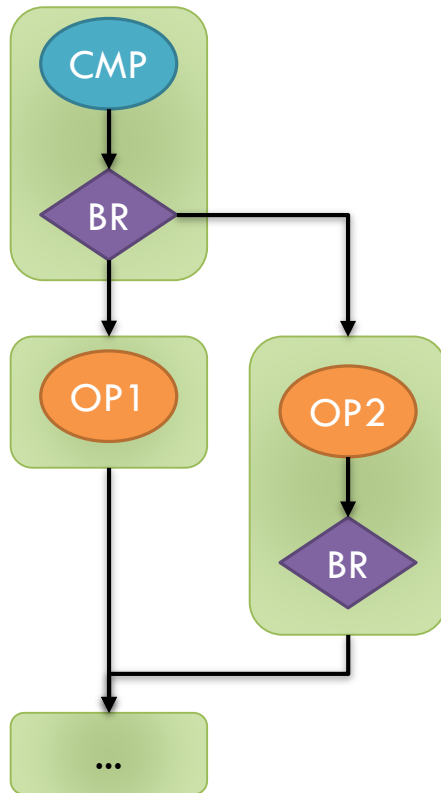
Uma operação ...

- Completada se o predicado é verdadeiro
- Não é completada se o predicado é falso

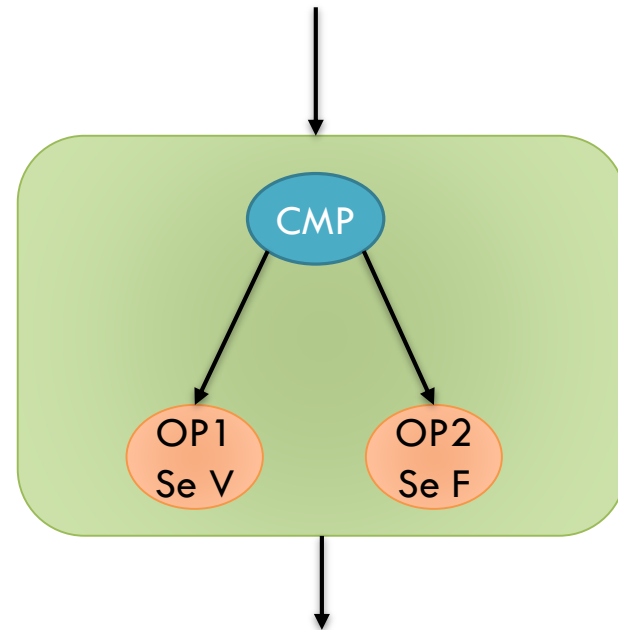
Esta técnica é conhecida como if-conversion

# EXEMPLO - ELIMINAÇÃO DE DESVIOS

**Programa original,**  
com instrução de branch BR



**Programa transformado,**  
com OP1 e OP2 sendo instruções  
com predicação



# MOVIMENTAÇÃO DE INSTRUÇÕES COM “PREDICAÇÃO”

A movimentação de instruções entre diferentes blocos básicos é suportada pela predicação de instruções

Através dessa técnica as instruções podem ser movimentadas sem a necessidade de se realizar execução especulativa

A movimentação de instruções permite ao compilador explorar melhor o paralelismo oferecido pelo processador

# PREDICAÇÃO - EXEMPLO

```
R1      <- R2 + R3
R4      <- M[X]
R5      <- R4 + R3
If R5 = 0, goto L
```

...  
...  
...  
...

```
R6      <- R2 * R3
R7      <- R1 * R3
M[X]    <- R7
R8      <- R4 + R3
R9      <- R6 + R4
```

## Com predicação

```
R1      ← R2 + R3
R4      ← M[X] (load)
R5      ← R4 + R3
P2    ← (R5 = 0)
R6      ← R2 * R3      if P2
R7      ← R1 * R3      if P2
M[X]    ← R7 (store)  if P2
R8      ← R4 + R3      if P2
R9      ← R6 + R4      if P2
```



# INSTRUÇÕES DE PREDICAÇÃO EM X86

X86 também possui instruções de predicação

São basicamente os CMOV (Conditional Move)

Então o compilador deve calcular o resultado em algum registrador temporário, e só mover para o registrador de destino diante da condicional.

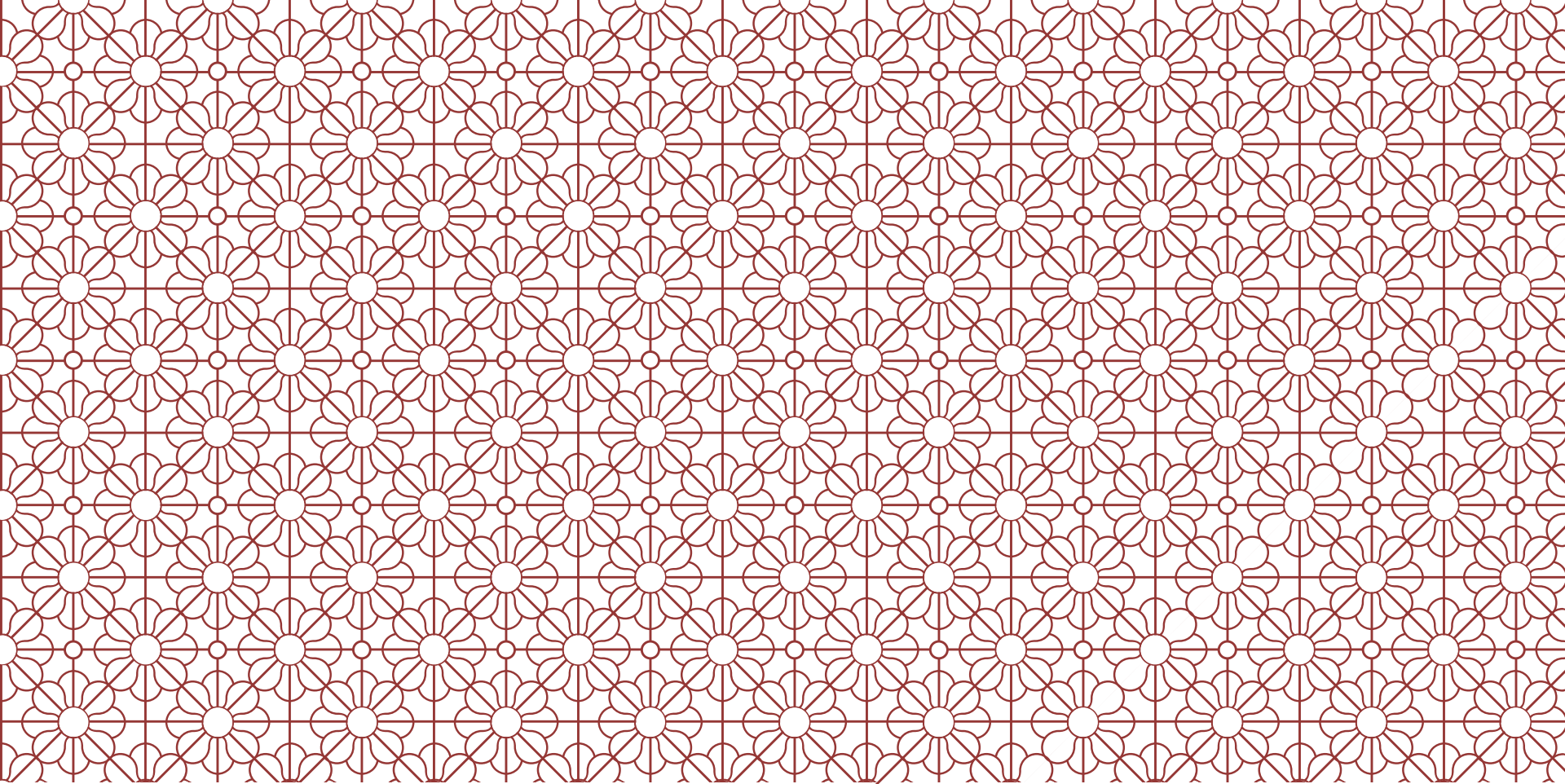
# INSTRUÇÕES DE PREDICAÇÃO EM X86

X86 também possui instruções de predicação **vetorial**

Também existem instruções com máscara de bits, comum em instruções vetoriais. Ex.

$A[0:15] = A[0:15] + B[0:15]$  (usando a máscara  $C[0:15]$ )

|                         |     |     |     |     |     |
|-------------------------|-----|-----|-----|-----|-----|
| <b>A</b>                | 10  | 20  | 30  | 40  | 50  |
| <b>MASK C</b>           | 0   | 1   | 1   | 0   | 1   |
| <b>B</b>                | 100 | 100 | 100 | 100 | 100 |
| <b>A Resultado Soma</b> | 10  | 120 | 130 | 40  | 150 |



# ESCALONAMIENTO CÍCLICO (LOOP UNROLLING / SOFTWARE PIPELINING)

# ESCALONAMENTO CÍCLICO

Código do ciclo inicial é reescrito sequencialmente até conseguir um padrão de instruções que se repetem

- Loop unrolling

Este padrão repetido ao longo do tempo é chamado de kernel (núcleo) do laço

O kernel do laço será realizado por uma (ou mais) instruções VLIW

Cada instrução VLIW pode conter operações de diferentes ciclos antes da duplicação (isto é representado com índices)

# ESCALONAMENTO CÍCLICO — EXEMPLO

R1  $\leftarrow$  2.14

R3  $\leftarrow$  X

R5  $\leftarrow$  Y

L: R2  $\leftarrow$  load M[R3++]

R4  $\leftarrow$  R2\*R1

store M [R5++]  $\leftarrow$  R4

if R3 < X+100 goto L

**Laço inicial**

# ESCALONAMENTO CÍCLICO — EXEMPLO

R1  $\leftarrow$  2.14  
R3  $\leftarrow$  X  
R5  $\leftarrow$  Y

R1  $\leftarrow$  2.14  
R3  $\leftarrow$  X  
R5  $\leftarrow$  Y

L: R2  $\leftarrow$  load M[R3++]  
R4  $\leftarrow$  R2\*R1  
store M[R5++]  $\leftarrow$  R4  
if R3 < X+100 goto L

**Laço inicial**

L: R2  $\leftarrow$  load M[R3++]  
R4  $\leftarrow$  R2\*R1  
store M[R5++]  $\leftarrow$  R4

R2  $\leftarrow$  load M[R3++]  
R4  $\leftarrow$  R2\*R1  
store M[R5++]  $\leftarrow$  R4

R2  $\leftarrow$  load M[R3++]  
R4  $\leftarrow$  R2\*R1  
store M[R5++]  $\leftarrow$  R4

R2  $\leftarrow$  load M[R3++]  
R4  $\leftarrow$  R2\*R1  
store M[R5++]  $\leftarrow$  R4

if R3 < X+100 goto L

**Laço reescrito 4 vezes**

Devemos escalonar as  
instruções sem  
dependência o mais  
cedo possível !

# ESCALONAMENTO CÍCLICO — EXEMPLO (2)

R1  $\leftarrow$  2.14

R3  $\leftarrow$  X

R5  $\leftarrow$  Y

L: R2  $\leftarrow$  load M [ R3++ ]  
R4  $\leftarrow$  R2\*R1  
store M [ R5++ ]  $\leftarrow$  R4

R2  $\leftarrow$  load M [ R3++ ]  
R4  $\leftarrow$  R2\*R1  
store M [ R5++ ]  $\leftarrow$  R4

R2  $\leftarrow$  load M [ R3++ ]  
R4  $\leftarrow$  R2\*R1  
store M [ R5++ ]  $\leftarrow$  R4

R2  $\leftarrow$  load M [ R3++ ]  
R4  $\leftarrow$  R2\*R1  
store M [ R5++ ]  $\leftarrow$  R4

if R3 < X+100 goto L

**Laço reescrito 4 vezes**

# ESCALONAMENTO CÍCLICO – EXEMPLO (2)

R1  $\leftarrow$  2.14

R3  $\leftarrow$  X

R5  $\leftarrow$  Y

L: R2  $\leftarrow$  load M [ R3++ ]  
R4  $\leftarrow$  R2\*R1  
store M [ R5++ ]  $\leftarrow$  R4

R2  $\leftarrow$  load M [ R3++ ]  
R4  $\leftarrow$  R2\*R1  
store M [ R5++ ]  $\leftarrow$  R4

R2  $\leftarrow$  load M [ R3++ ]  
R4  $\leftarrow$  R2\*R1  
store M [ R5++ ]  $\leftarrow$  R4

R2  $\leftarrow$  load M [ R3++ ]  
R4  $\leftarrow$  R2\*R1  
store M [ R5++ ]  $\leftarrow$  R4

if R3 < X+100 goto L

R1  $\leftarrow$  2.14

R3  $\leftarrow$  X

R5  $\leftarrow$  Y

R2<sub>1</sub>  $\leftarrow$  load M[R3++]

R4<sub>1</sub>  $\leftarrow$  R2<sub>1</sub>\*R1

store M[R5++]  $\leftarrow$  R4<sub>1</sub>

1 laço

**Laço reescrito 4 vezes**



# ESCALONAMENTO CÍCLICO – EXEMPLO (2)

R1  $\leftarrow$  2.14

R3  $\leftarrow$  X

R5  $\leftarrow$  Y

L: R2  $\leftarrow$  load M [ R3++ ]  
R4  $\leftarrow$  R2\*R1  
store M [ R5++ ]  $\leftarrow$  R4

R2  $\leftarrow$  load M [ R3++ ]  
R4  $\leftarrow$  R2\*R1  
store M [ R5++ ]  $\leftarrow$  R4

R2  $\leftarrow$  load M [ R3++ ]  
R4  $\leftarrow$  R2\*R1  
store M [ R5++ ]  $\leftarrow$  R4

R2  $\leftarrow$  load M [ R3++ ]  
R4  $\leftarrow$  R2\*R1  
store M [ R5++ ]  $\leftarrow$  R4

if R3 < X+100 goto L

R1  $\leftarrow$  2.14

R3  $\leftarrow$  X

R5  $\leftarrow$  Y

R2<sub>1</sub>  $\leftarrow$  load M[R3++]

R2<sub>2</sub>  $\leftarrow$  load M[R3++]

R4<sub>1</sub>  $\leftarrow$  R2<sub>1</sub>\*R1

R4<sub>2</sub>  $\leftarrow$  R2<sub>2</sub>\*R1

store M[R5++]  $\leftarrow$  R4<sub>1</sub>  
store M[R5++]  $\leftarrow$  R4<sub>2</sub>

1 e 2º laço

**Estamos  
simplesmente  
escalando  
as instruções,  
olhando as  
dependências**

**Laço reescrito 4 vezes**

# ESCALONAMENTO CÍCLICO – EXEMPLO (2)

R1  $\leftarrow$  2.14

R3  $\leftarrow$  X

R5  $\leftarrow$  Y

L: R2  $\leftarrow$  load M [ R3++ ]  
 R4  $\leftarrow$  R2\*R1  
 store M [ R5++ ]  $\leftarrow$  R4

R2  $\leftarrow$  load M [ R3++ ]  
 R4  $\leftarrow$  R2\*R1  
 store M [ R5++ ]  $\leftarrow$  R4

R2  $\leftarrow$  load M [ R3++ ]  
 R4  $\leftarrow$  R2\*R1  
 store M [ R5++ ]  $\leftarrow$  R4

R2  $\leftarrow$  load M [ R3++ ]  
 R4  $\leftarrow$  R2\*R1  
 store M [ R5++ ]  $\leftarrow$  R4

if R3 < X+100 goto L

R1  $\leftarrow$  2.14

R3  $\leftarrow$  X

R5  $\leftarrow$  Y

R2<sub>1</sub>  $\leftarrow$  load M[R3++]

R2<sub>2</sub>  $\leftarrow$  load M[R3++]

R2<sub>3</sub>  $\leftarrow$  load M[R3++]

...

R4<sub>1</sub>  $\leftarrow$  R2<sub>1</sub>\*R1

R4<sub>2</sub>  $\leftarrow$  R2<sub>2</sub>\*R1

R4<sub>3</sub>  $\leftarrow$  R2<sub>3</sub>\*R1

...

store M[R5++]  $\leftarrow$  R4<sub>1</sub>

store M[R5++]  $\leftarrow$  R4<sub>2</sub>

store M[R5++]  $\leftarrow$  R4<sub>3</sub>

...

1 ~ 3º laço

**Estamos  
enchendo o  
“Pipeline”  
em software!**

**Laço reescrito 4 vezes**

# ESCALONAMENTO CÍCLICO – EXEMPLO (2)

R1  $\leftarrow$  2.14

R3  $\leftarrow$  X

R5  $\leftarrow$  Y

L: R2  $\leftarrow$  load M [ R3++ ]  
 R4  $\leftarrow$  R2\*R1  
 store M [ R5++ ]  $\leftarrow$  R4

R2  $\leftarrow$  load M [ R3++ ]  
 R4  $\leftarrow$  R2\*R1  
 store M [ R5++ ]  $\leftarrow$  R4

R2  $\leftarrow$  load M [ R3++ ]  
 R4  $\leftarrow$  R2\*R1  
 store M [ R5++ ]  $\leftarrow$  R4

R2  $\leftarrow$  load M [ R3++ ]  
 R4  $\leftarrow$  R2\*R1  
 store M [ R5++ ]  $\leftarrow$  R4

if R3 < X+100 goto L

R1  $\leftarrow$  2.14

R3  $\leftarrow$  X

R5  $\leftarrow$  Y

L:

R2<sub>1</sub>  $\leftarrow$  load M[R3++]

R2<sub>2</sub>  $\leftarrow$  load M[R3++]

R2<sub>3</sub>  $\leftarrow$  load M[R3++]

R2<sub>4</sub>  $\leftarrow$  load M[R3++]

R2<sub>5</sub>  $\leftarrow$  load M[R3++]

R2<sub>6</sub>  $\leftarrow$  load M[R3++]

R2<sub>7</sub>  $\leftarrow$  load M[R3++]

R2<sub>8</sub>  $\leftarrow$  load M[R3++]

R2<sub>9</sub>  $\leftarrow$  load M[R3++]

R2<sub>10</sub>  $\leftarrow$  load M[R3++]

R4<sub>1</sub>  $\leftarrow$  R2<sub>1</sub>\*R1

R4<sub>2</sub>  $\leftarrow$  R2<sub>2</sub>\*R1

R4<sub>3</sub>  $\leftarrow$  R2<sub>3</sub>\*R1

R4<sub>4</sub>  $\leftarrow$  R2<sub>4</sub>\*R1

R4<sub>5</sub>  $\leftarrow$  R2<sub>5</sub>\*R1

R4<sub>6</sub>  $\leftarrow$  R2<sub>6</sub>\*R1

R4<sub>7</sub>  $\leftarrow$  R2<sub>7</sub>\*R1

R4<sub>8</sub>  $\leftarrow$  R2<sub>8</sub>\*R1

R4<sub>9</sub>  $\leftarrow$  R2<sub>9</sub>\*R1

R4<sub>10</sub>  $\leftarrow$  R2<sub>10</sub>\*R1

store M[R5++]  $\leftarrow$  R4<sub>1</sub>

store M[R5++]  $\leftarrow$  R4<sub>2</sub>

store M[R5++]  $\leftarrow$  R4<sub>3</sub>

store M[R5++]  $\leftarrow$  R4<sub>4</sub>

store M[R5++]  $\leftarrow$  R4<sub>5</sub>

store M[R5++]  $\leftarrow$  R4<sub>6</sub>

store M[R5++]  $\leftarrow$  R4<sub>7</sub>

store M[R5++]  $\leftarrow$  R4<sub>8</sub>

store M[R5++]  $\leftarrow$  R4<sub>9</sub>

store M[R5++]  $\leftarrow$  R4<sub>10</sub>

if R3 < X+100 goto L

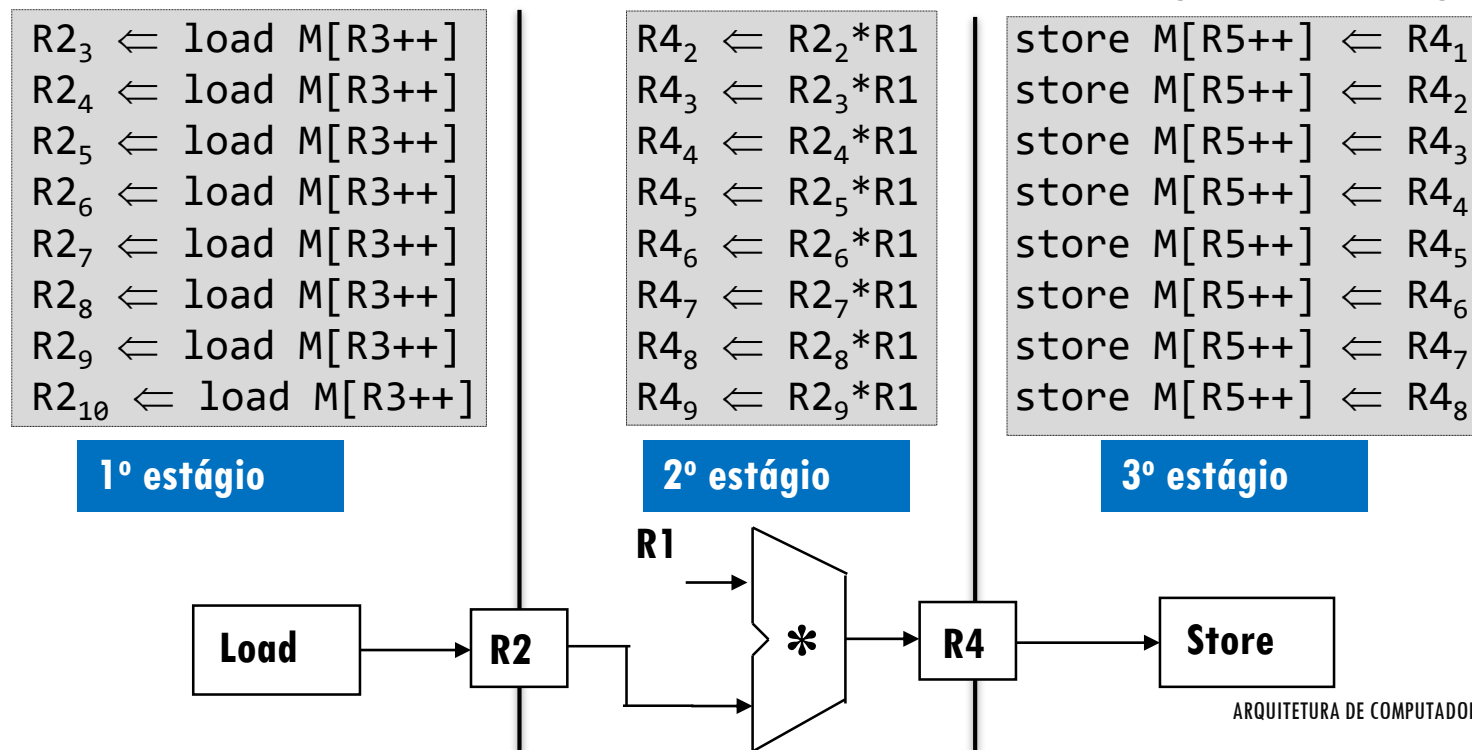
**Ao longo do tempo estas três operações podem sempre ser executadas em paralelo**

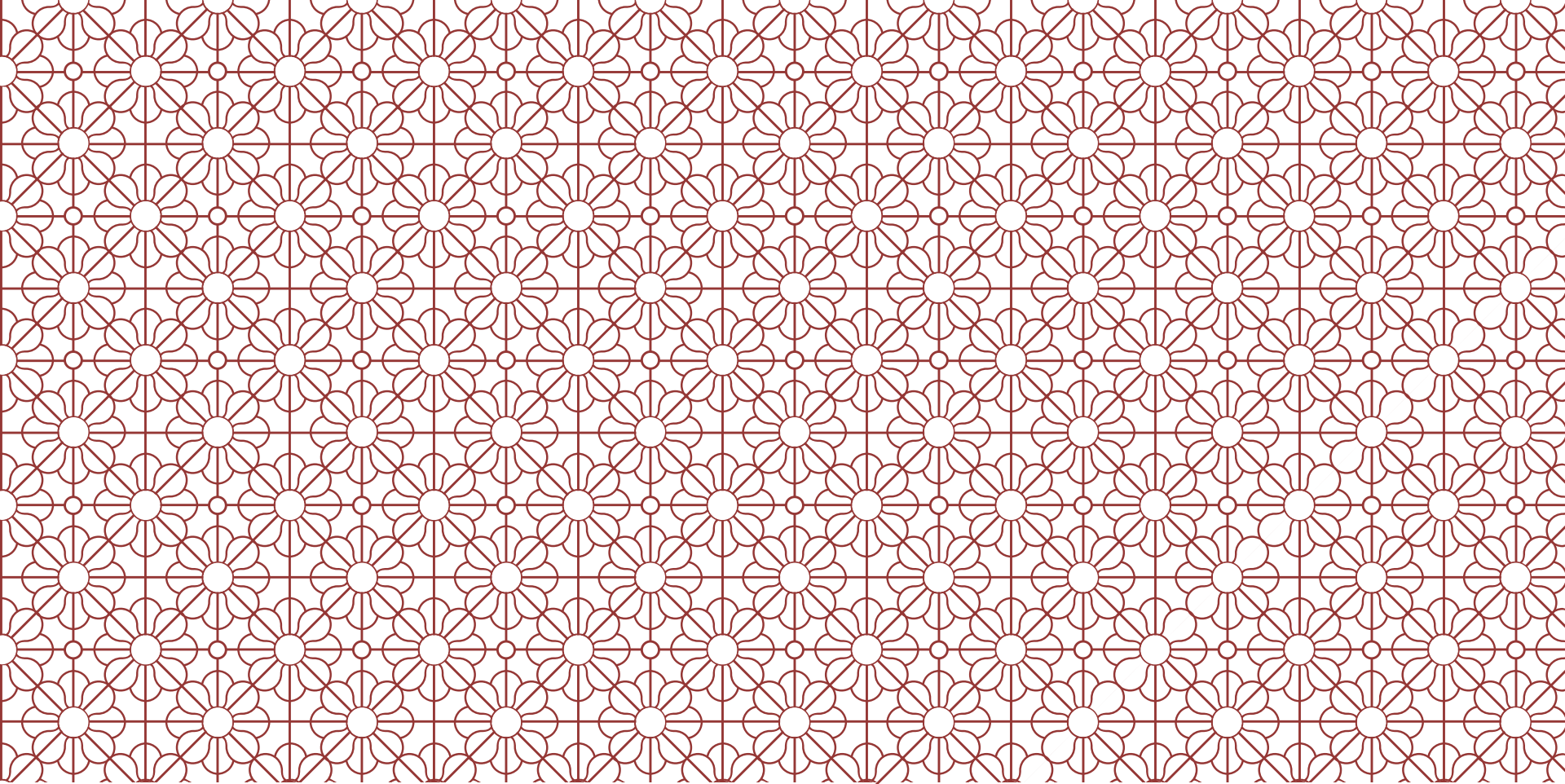
**Laço reescrito 4 vezes**

# ESCALONAMENTO CÍCLICO – EXEMPLO (3)

Ao longo do tempo estas três operações podem sempre ser executadas em paralelo

Na prática o compilador usou as unidades operativas para fazer (via software) um pipeline de 3 estágios dedicado à execução deste laço





# PROCESSADOR ITANIUM

# PROCESSADOR ITANIUM

Itanium é o primeiro processador projetado com a tecnologia **IA64**, **≠** X86\_64 uma arquitetura VLIW da Intel+HP

- Atualmente a ISA é chamada de Intel Itanium Architecture

O compilador agrupa e escalona as instruções com base na filosofia EPIC (Explicitly Parallel Instruction Computing)

O primeiro Itanium (Merced), foi lançado em 2001

A última versão (Kittson), foi lançado em Fev/2017

# VISÃO CONCEITUAL DO ITANIUM

Compiler-programmed features:

|              |                                             |                          |             |                              |              |
|--------------|---------------------------------------------|--------------------------|-------------|------------------------------|--------------|
| Branch hints | Explicit parallelism; instruction templates | Register stack, rotation | Predication | Data and control speculation | Memory hints |
|--------------|---------------------------------------------|--------------------------|-------------|------------------------------|--------------|

Hardware features:

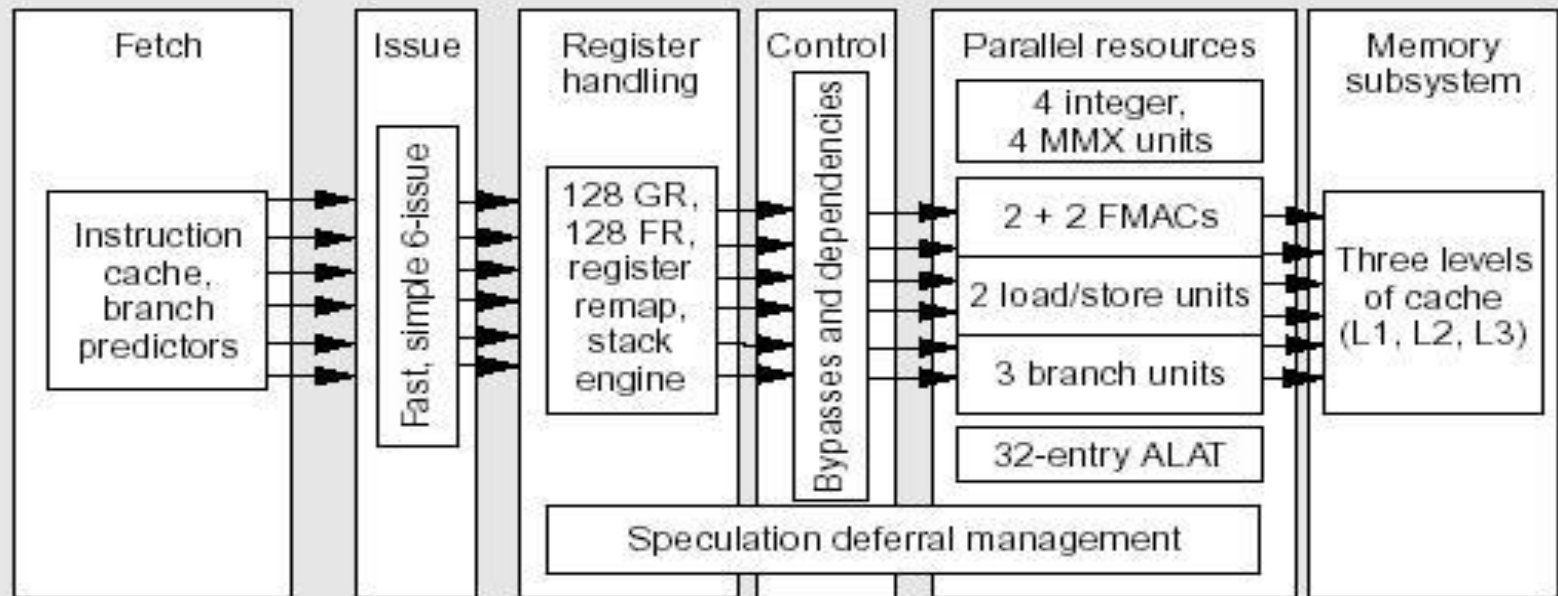
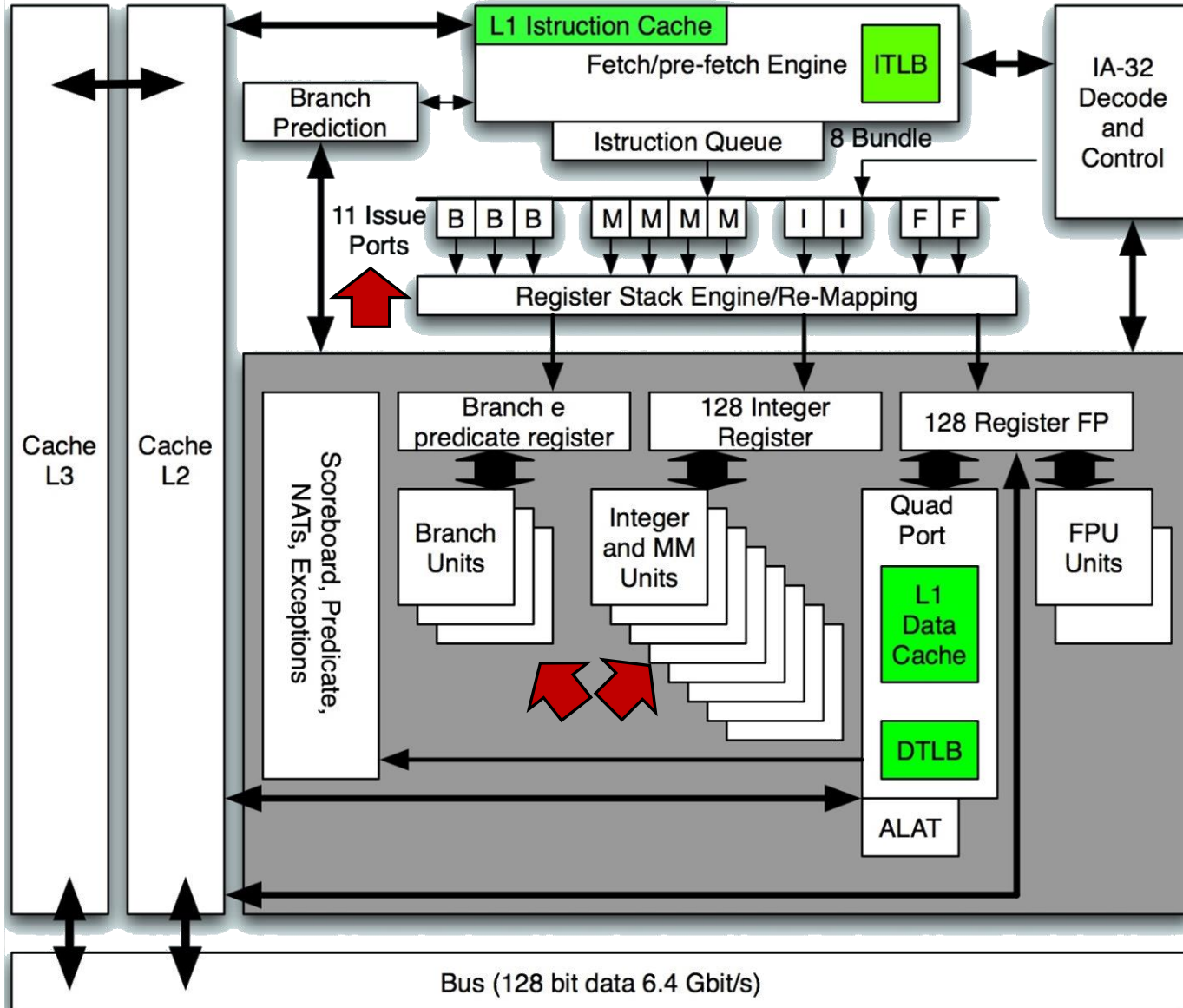
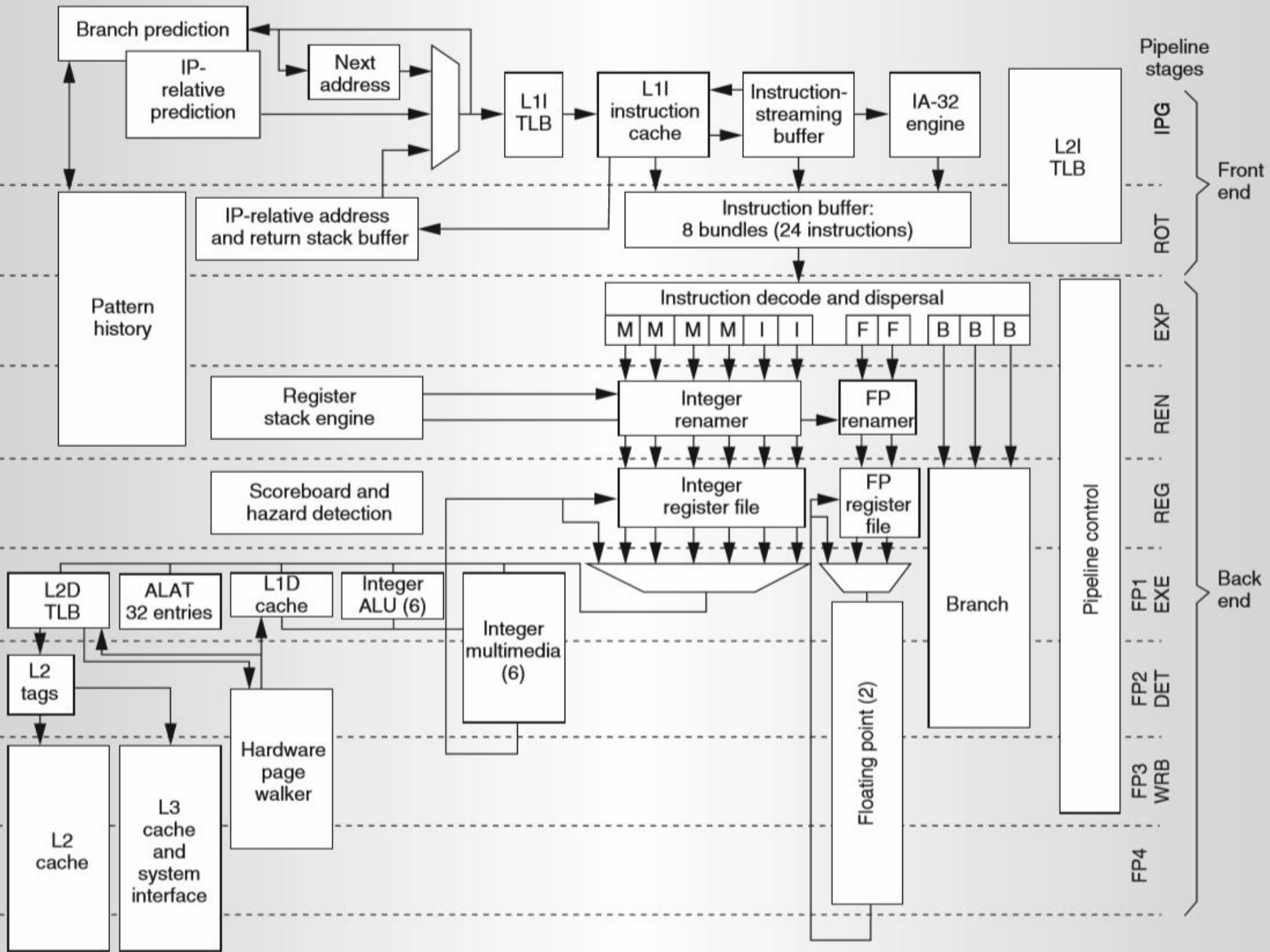


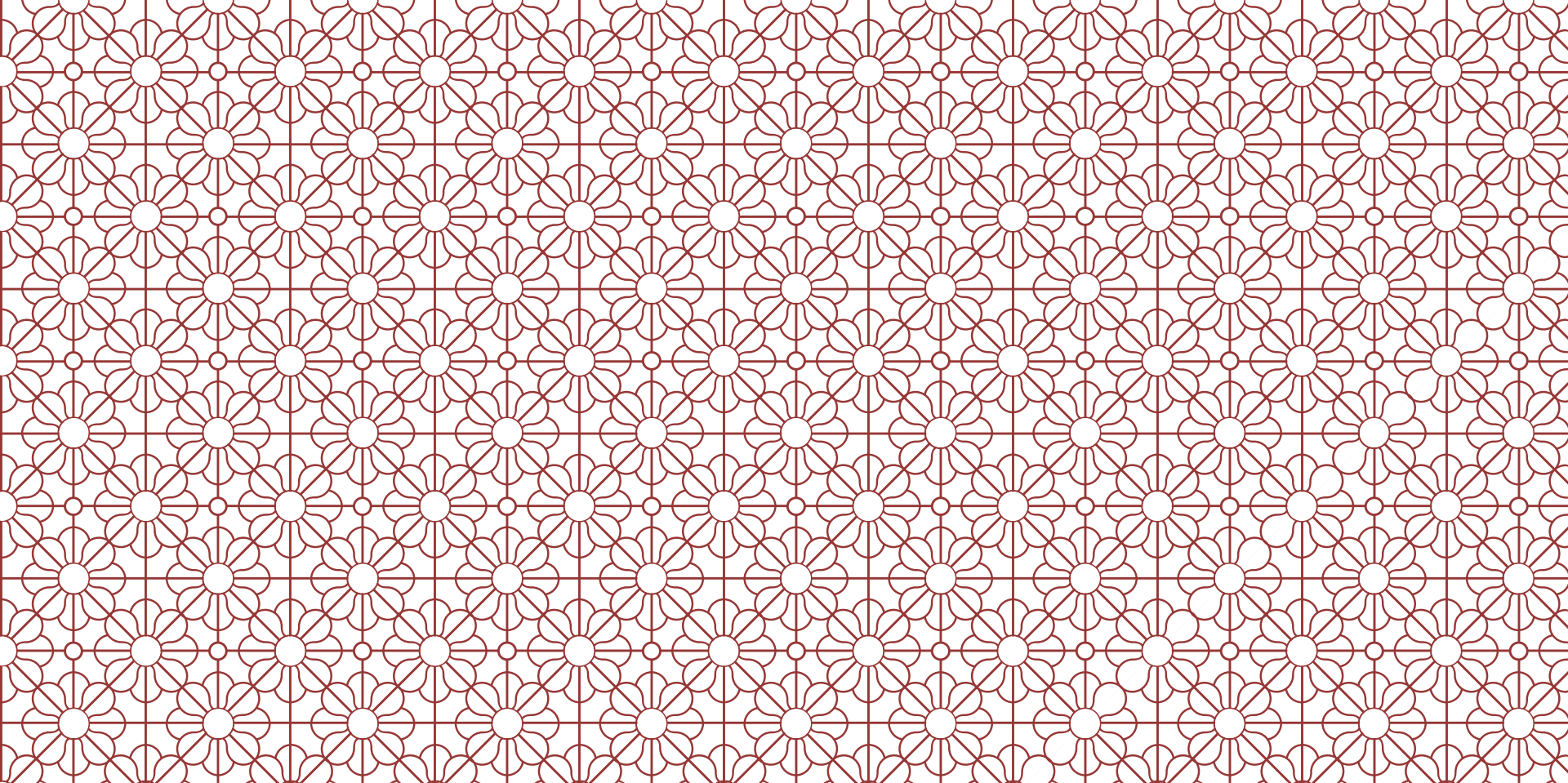
Figure 1. Conceptual view of EPIC hardware. GR: general register file; FR: floating-point register file

# PROCESSADOR ITANIUM



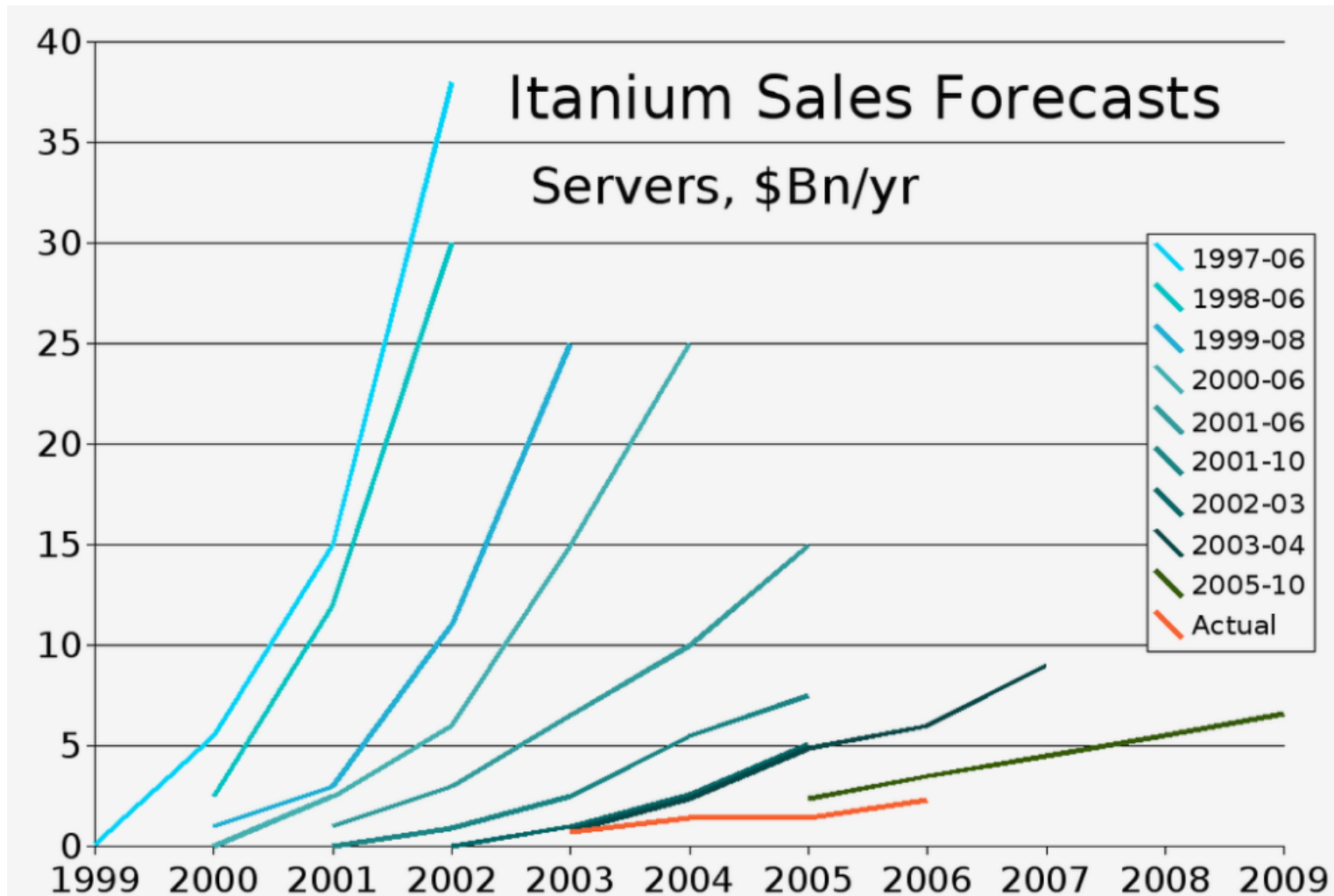






# QUAL O FUTURO DO ITANIUM

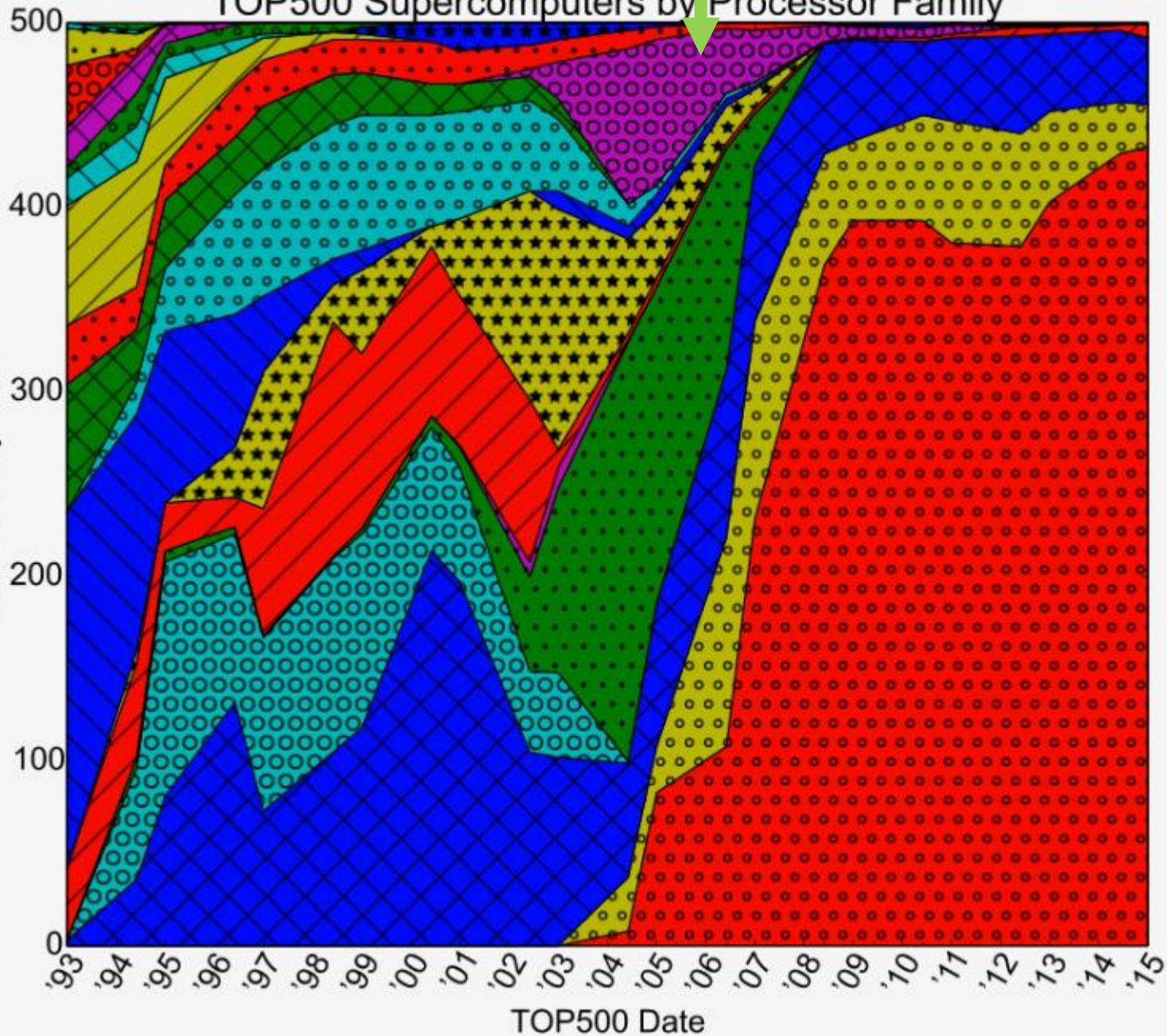
# PREVISÃO DE VENDAS VS. REALIDADE





# TOP500 Supercomputers by Processor Family

Number of systems



- x86-64 (Intel)
- x86-64 (AMD)
- POWER
- MIPS
- x86-32 (Intel)
- x86-32 (AMD)
- Sparc
- PA-RISC
- Cray
- Alpha
- Fujitsu
- Itanium (Intel)
- NEC
- Intel i860
- Hitachi SR8000
- TMC CM2
- Hitachi
- KSR
- Convex
- Maspar
- Others
- nCube
- IBM3090

# INTEL ITANIUM ★ 2001 - † 2017



**“[Itanium] will be remembered as one of those interesting architectural experiments. It lived a lot longer than others,”  
Dean McCarron (Mercury Research)**